Chapter 17 — JavaScript for Interviews

A JavaScript interview usually has three parts. A **coding round** asks you to solve an algorithm problem (in JavaScript, so you need to know which built-ins are fast and which are traps). A **"build it yourself" round** asks you to write a utility that libraries normally give you — `debounce`, `Promise.all`, a deep clone. And a **language round** throws short snippets at you and asks "what does this print, and why?". This chapter prepares all three. It does not re-teach the language: every answer links to the section of this book that explains it fully.

What you'll learn

1. The JavaScript **DSA toolkit**: arrays as stacks and queues, `Map` / `Set`, sorting comparators and stability, number pitfalls (`0.1 + 0.2`, safe integers, integer division), building strings, `BigInt` — with a complexity table.
2. **Twenty-one coding patterns** — two pointers, sliding window, hashing, stack, prefix sums, intervals, heap (with a hand-written `MinHeap`, since JavaScript has none, plus two heaps for a running median and k-way merge), binary search, BFS, DFS, dynamic programming with memoization, backtracking, linked lists, fast & slow pointers, trees, tries, topological sort and union-find, greedy, sort & search (quickselect, Dutch flag, merge sort), bit manipulation, math & geometry — each with two or three classic problems in runnable, asserted code.
3. **Implement-from-scratch classics** with tests: `debounce`, `throttle`, `curry`, `compose` / `pipe`, `memoize`, `flatten`, a `bind` polyfill, deep clone (`structuredClone` vs manual), deep equal, an LRU cache, an `EventEmitter`, and `Promise.all` / `allSettled` / `race` / `any` polyfills.
4. **Fifty-three tricky questions** with real outputs: coercion, equality, `typeof null`, `NaN`, hoisting and the TDZ, closures in loops, `this`, arrow vs regular functions, prototypes, classes, event-loop ordering puzzles, shallow vs deep copies, garbage collection and memory leaks.

Before you start. This chapter assumes the whole book up to Chapter 16. The tests use `node:assert/strict` and fake timers from `node:test`, explained in §16.2 and §16.5. The same patterns are solved in Python in the Python Handbook's Chapter 19: Python for Coding Interviews — handy for comparing; here we focus on what is specific to JavaScript.

All code is in `code/`. Every program checks itself with `assert` and exits with an error if any answer is wrong; outputs below are real, from Node 24.

File	Contents
`01_toolkit.mjs`	Arrays, queue, grids, Map/Set, sorting, numbers, strings, BigInt
`02_patterns_arrays.mjs`	Two pointers, sliding window, hashing, stack, prefix sums, intervals
`03_patterns_search_graphs_dp.mjs`	MinHeap, two heaps, k-way merge, binary search, BFS, DFS, DP, backtracking
`04_classics_functions.mjs`	debounce, throttle, curry, compose/pipe, memoize, flatten, bind/call
`05_classics_data.mjs`	structuredClone, deepClone, deepEqual, LRU cache, EventEmitter
`06_promise_polyfills.mjs`	all/allSettled/race/any, timeout, promisify, pool, retry
`07_qa_language.js`	Outputs for the language questions (CommonJS, sloppy mode on purpose)
`08_qa_async_and_memory.mjs`	Outputs for event-loop, strict-mode and memory questions
`09_patterns_lists_trees_tries.mjs`	Linked lists, fast & slow pointers, trees, tries (Patterns 13–16)
`10_patterns_graphs_greedy_bits_math.mjs`	Topological sort, union-find, greedy, quickselect/Dutch flag/merge sort, bits, math & geometry (Patterns 17–21)

17.1 The JavaScript DSA toolkit

What it is

DSA means *data structures and algorithms*. In an interview you have to turn an idea ("use a queue", "count with a hash map") into working code quickly. JavaScript gives you fewer ready-made structures than Java or Python — no priority queue, no deque, no sorted map — so you need to know what the built-ins really cost and how to fill the gaps.

Why it matters

Two JavaScript-specific traps sink otherwise-correct solutions: **hidden O(n) operations** (`shift`, `unshift`, `includes` on an array inside a loop) and **wrong answers from defaults** (`sort()` comparing numbers as text, floating-point arithmetic). Knowing the table below is the difference between an O(n) and an O(n²) solution.

Complexity of the built-ins

Operation	Cost	Note
`arr[i]`, `arr[i] = x`, `arr.length`, `arr.at(-1)`	O(1)	
`push`, `pop`	O(1) amortized	Use the end of an array as a **stack**
`shift`, `unshift`, `splice(i, …)`	O(n)	Every later element moves — avoid in loops
`indexOf`, `includes`, `find`, `some`	O(n)	Put values in a `Set` if you look them up repeatedly
`slice`, spread `[...a]`, `concat`, `Array.from`	O(k)	k = number of elements copied
`map`, `filter`, `reduce`, `forEach`	O(n)	Plus the callback's cost
`sort`, `toSorted`	O(n log n)	Stable (guaranteed since ES2019); V8 uses TimSort
`Map` / `Set` `get`, `set`, `has`, `add`, `delete`	O(1) average	The spec only requires *sublinear* on average; engines use hash tables
Object property read/write, `in`, `Object.hasOwn`	O(1) average	
`Object.keys` / `entries`, `[...map]`	O(n)	
`str[i]`, `str.length`, `charCodeAt`	O(1)	Strings are immutable
`str.slice`, `split`, `join`, `[...str]`	O(n)	
`JSON.stringify`, `structuredClone`	O(size)	
Heap push/pop (you write it, §17.2)	O(log n)	

Arrays as stack, queue and grid

From `code/01_toolkit.mjs`:

```
const stack = [];
stack.push(1); stack.push(2);
console.log('stack pop:', stack.pop(), 'peek:', stack.at(-1));   // O(1) at the end

const q = [1, 2, 3];
console.log('queue shift:', q.shift(), 'left:', q);             // shift is O(n)!

// A real O(1) queue: keep a head index instead of shifting
class Queue {
  #items = []; #head = 0;
  push(x) { this.#items.push(x); }
  shift() {
    if (this.#head === this.#items.length) return undefined;
    const x = this.#items[this.#head++];
    if (this.#head > 1024 && this.#head * 2 > this.#items.length) {   // compact occasionally
      this.#items = this.#items.slice(this.#head); this.#head = 0;
    }
    return x;
  }
  get size() { return this.#items.length - this.#head; }
}

// 2D grid: Array.from, NOT fill with an array (shared rows!)
const bad = new Array(2).fill(new Array(2).fill(0));
bad[0][0] = 9;
const grid = Array.from({ length: 2 }, () => new Array(2).fill(0));
grid[0][0] = 9;
```

Output:

```
stack pop: 2 peek: 1
queue shift: 1 left: [ 2, 3 ]
Queue: a 1
fill(array) shares rows: [[9,0],[9,0]]
Array.from rows are separate: [[9,0],[0,0]]
```

1. A **stack** (last in, first out) is just `push` / `pop`; peek with `at(-1)`.
2. A **queue** (first in, first out) with `shift()` is fine for small inputs, but in general each `shift` moves every element. Measured in Node 24: emptying a 10,000-element array with `shift` took 0.3 ms, a 100,000-element one 0.6 s, a 400,000-element one 10 s — quadratic growth (V8 has a shortcut for smaller arrays, not for big ones). For BFS on large graphs keep a **head index** — the `Queue` class above, or simply `for (let head = 0; head < queue.length; head++)` as in §17.2.
3. `fill(x)` puts the *same* value in every slot; with an array as `x`, every row is the same object. `Array.from({ length }, () => …)` calls the function once per row. (Sorting, copying and array methods are taught in Chapter 7.)

`Map` and `Set`

```
const freq = new Map();
for (const ch of 'banana') freq.set(ch, (freq.get(ch) ?? 0) + 1);
const visited = new Set();
visited.add(`${1},${2}`);                       // compound key as a string
new Set([[1, 2]]).has([1, 2]);                  // false: arrays compare by reference
const a = new Set([1, 2, 3]), b = new Set([2, 3, 4]);
a.union(b); a.intersection(b); a.difference(b);     // ES2025
```

Output:

```
freq: [ [ 'b', 1 ], [ 'a', 3 ], [ 'n', 2 ] ]
set: [ 3, 1, 2 ] has 2? true
visited has "1,2"? true | Set of arrays: false
union [ 1, 2, 3, 4 ] intersection [ 2, 3 ] difference [ 1 ]
```

- Prefer `Map` over a plain object for counting and lookups: any key type, no inherited keys like `constructor`, a real `size`, and insertion-order iteration (§8.1).
- `Map` and `Set` compare keys with **SameValueZero** (like `===`, except `NaN` equals `NaN`). Two arrays with equal contents are *different* keys, so encode coordinates as a string (`` `${r},${c}` ``) or a number (`r * cols + c`).
- The ES2025 set methods (`union`, `intersection`, `difference`, `symmetricDifference`, `isSubsetOf`, …) are in Node 22+ and all current browsers (§8.2).

Sorting: comparators and stability

```
[10, 9, 1, 100].sort();                          // compared as STRINGS
[10, 9, 1, 100].sort((x, y) => x - y);           // numeric
people.toSorted((p, r) => p.age - r.age);        // stable: ties keep input order
people.toSorted((p, r) => r.age - p.age || p.name.localeCompare(r.name));  // two keys
```

Output:

```
default sort: [ 1, 10, 100, 9 ]
numeric sort: [ 1, 9, 10, 100 ]
by age (stable): Bob Dee Ann Cid
age desc, name asc: Ann Cid Bob Dee
original untouched: Ann Bob Cid Dee
```

1. A **comparator** `(a, b) => number` returns negative if `a` goes first, positive if `b` goes first, `0` for a tie. `a - b` sorts numbers ascending; `b - a` descending.
2. **Multi-key sort**: chain comparisons with `||` — when the first difference is `0`, the second decides.
3. **Stability** means equal elements keep their original relative order (Bob before Dee, Ann before Cid above). It is guaranteed since ES2019, so you can sort by a secondary key first and the primary key second.
4. `sort` **mutates**; `toSorted` (ES2023) returns a new array. In an interview, say which one you use — mutating the caller's input is a classic review comment.
5. Common mistake: returning a boolean (`(a, b) => a > b`). `true`/`false` become `1`/`0`, never negative, so the result is engine-dependent and wrong. Full details: §7.6.

Number pitfalls

```
0.1 + 0.2;                          // 0.30000000000000004
Number.MAX_SAFE_INTEGER;            // 9007199254740991 = 2**53 - 1
2 ** 53 + 1;                        // 9007199254740992 (can't be represented)
Math.trunc(-7 / 2); Math.floor(-7 / 2);   // -3 and -4
-7 % 3;                             // -1 (sign follows the dividend)
(2 ** 31 + 2) >> 1;                 // -1073741823: bit operators use 32-bit ints
Math.max(...[]);                    // -Infinity
```

Output:

```
0.1 + 0.2 = 0.30000000000000004 | === 0.3? false
close(0.1+0.2, 0.3): true
MAX_SAFE_INTEGER: 9007199254740991 | 2**53 + 1 = 9007199254740992
isSafeInteger(2**53): false
7 / 2 = 3.5 | Math.trunc: 3 | -7/2 trunc: -3 floor: -4
-7 % 3 = -1 | true modulo: 2
mid without overflow worry: 5 | but >> breaks above 2**31: -1073741823
Infinity sentinels: Infinity -Infinity
Math.max(...[]) = -Infinity | parseInt("08") = 8 | Number("") = 0
NaN === NaN: false | Number.isNaN(NaN): true | isNaN("x"): true Number.isNaN("x"): false
```

Pitfall	What to do
All numbers are 64-bit floats (IEEE 754 doubles), so `0.1 + 0.2 !== 0.3`	Compare with a tolerance (`Math.abs(a - b) < Number.EPSILON * scale`), or work in integers (cents, not dollars)
No integer division	`Math.trunc(a / b)` (towards zero) or `Math.floor(a / b)` (towards −∞) – they differ for negatives
`%` is a *remainder*, sign follows the left side	For array wrap-around use `((i % n) + n) % n`
Integers are exact only up to `2**53 - 1`	Check with `Number.isSafeInteger`; use `BigInt` beyond (e.g. factorials, large Fibonacci, 64-bit IDs)
Bitwise operators (`` `, >>, ~~` ``) convert to 32-bit signed ints	
`Infinity` / `-Infinity`	Handy initial values for min/max; but `Math.max(...[])` is `-Infinity` and spreading a very large array (about 125,000+ elements in Node 24 on this machine – the limit depends on stack size) throws `RangeError` – use a loop or `reduce`
`isNaN('x')` coerces first	Use `Number.isNaN`

Background: §2.10 Numbers in depth.

Strings: immutable, so build with arrays

```
const parts = [];
for (let i = 0; i < 5; i++) parts.push(i);
parts.join('-');                          // '0-1-2-3-4'
'd'.charCodeAt(0) - 97;                   // 3: letter -> index 0..25
[...'hello'].reverse().join('');          // 'olleh'
```

- Strings can't be changed in place; `s[0] = 'x'` silently does nothing (throws in strict mode). To edit, convert to an array (`[...s]`), change it, `join('')`.
- Building with `+=` in a loop is fine in modern engines (V8 uses *ropes*, a tree of pieces, to avoid copying), but `push` + `join` is the clear, portable idiom and makes the O(n) cost obvious.
- A 26-slot counting array (`charCodeAt(0) - 97`) is faster than a `Map` for lowercase-letter problems.
- `[...s]` splits by **code point** (emoji stay whole); `s.split('')` splits by UTF-16 code unit. See §6.5.

BigInt

```
const big = 2n ** 64n;                    // 18446744073709551616n, typeof 'bigint'
const factorial = (n) => (n <= 1n ? 1n : n * factorial(n - 1n));
factorial(25n);                           // 15511210043330985984000000n
7n / 2n;                                  // 3n (division truncates)
1n == 1; 1n === 1;                        // true, false
1n + 1;                                   // TypeError: can't mix BigInt and Number
JSON.stringify(1n);                       // TypeError
```

`BigInt` (ES2020) holds integers of any size — write a literal with an `n` suffix or call `BigInt(x)`. It has no fractions, no `Math.*` support, can't be mixed with numbers in arithmetic (convert explicitly), and is slower than numbers, so use it only when values may pass `2**53`.

In the interview

> **30-second answer — "What should I watch for when doing algorithms in JavaScript?"** Arrays are stacks (`push` / `pop`); queues need a head index, not `shift`. `Map` / `Set` give O(1) lookups (string keys for coordinates). Always pass `sort` a comparator; it mutates. Numbers are doubles: `Math.trunc` for integer division, `%` can be negative, exact only to $2^{53} - 1$ (then `BigInt`). No heap — write one.

- *Is `Array.prototype.sort` stable?* Yes, required since ES2019.
- *What does `sort()` do with no comparator?* Converts to strings and compares UTF-16 code units, with `undefined` last — so `[10, 9]` stays `[10, 9]`.
- *Why a `Map` rather than `{}` for counting?* Any key type, no prototype keys (`'constructor' in {}` is `true`), `size`, insertion order, optimised for frequent adds/removes.

Self-check

1. What is the cost of `arr.shift()`? → O(n): every remaining element moves one slot.
2. How do you create a 3×3 grid of zeros safely? → `Array.from({ length: 3 }, () => new Array(3).fill(0))`.
3. How do you sort strings alphabetically in a locale-aware way? → `arr.sort((a, b) => a.localeCompare(b))`.
4. Is `Number.isSafeInteger(2 ** 53)` true? → No — the largest safe integer is `2 ** 53 - 1`.

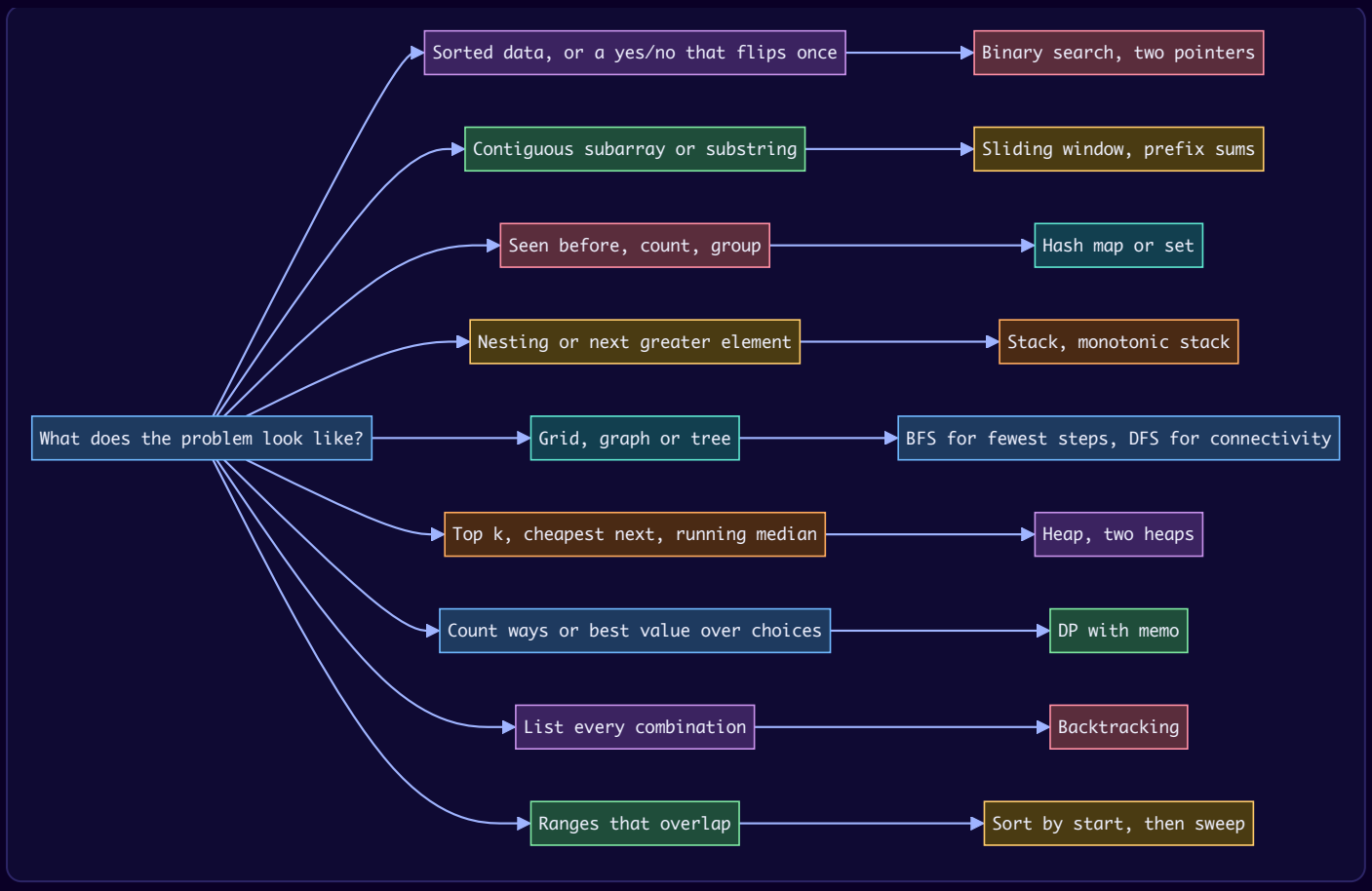
TL;DR

Stack = `push` / `pop`; queue = head index; grid = `Array.from`. `Map` / `Set` for O(1) lookups (string keys for pairs). Always give `sort` a comparator; it's stable; `toSorted` doesn't mutate. Floats: tolerance for equality, `Math.trunc` for division, safe up to $2^{53} - 1$, `BigInt` beyond.

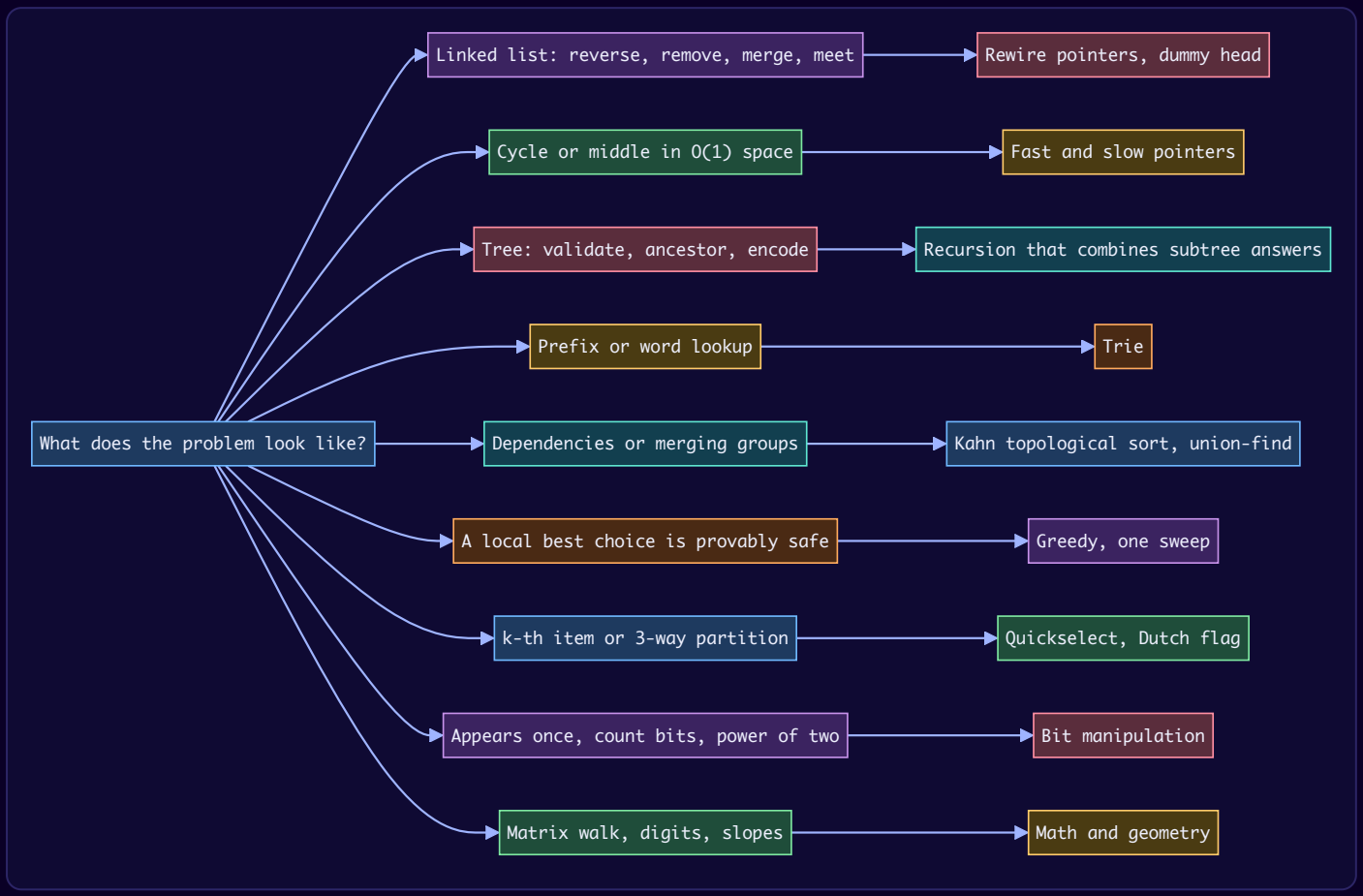
17.2 Coding patterns

What it is

A **pattern** is a reusable problem-solving shape. Most interview problems are one of about twenty patterns in disguise; recognising the pattern gives you the algorithm and its complexity almost for free. The decision chart below is a starting point, not a law.



Patterns 13–21 cover the remaining shapes – specific data structures and a few algorithm families:



For each pattern: the idea, when it applies, the code (with asserts), and its cost. The full programs are `code/02_patterns_arrays.mjs`, `code/03_patterns_search_graphs_dp.mjs`, `code/09_patterns_lists_trees_tries.mjs` and `code/10_patterns_graphs_greedy_bits_math.mjs`.

Pattern 1 — Two pointers

Idea: two indexes move towards each other (or in the same direction) so each element is looked at once. **Signal:** sorted array, pairs, palindromes, in-place partitioning. **Cost:** O(n) time, O(1) space.

```
function pairSum(sorted, target) {
  let lo = 0, hi = sorted.length - 1;
  while (lo < hi) {
    const s = sorted[lo] + sorted[hi];
    if (s === target) return [lo, hi];
    if (s < target) lo++; else hi--;      // too small: need a bigger left value
  }
  return null;
}
assert.deepEqual(pairSum([1, 3, 4, 6, 9], 12), [1, 4]);

function isPalindrome(s) {
  const t = s.toLowerCase().replace(/[^a-z0-9]/g, '');
  for (let i = 0, j = t.length - 1; i < j; i++, j--) if (t[i] !== t[j]) return false;
  return true;
}
assert.equal(isPalindrome('A man, a plan, a canal: Panama'), true);
```

Why it's correct: if the sum is too small, the smallest element can't be part of any answer with a *smaller* partner, so discarding it (`lo++`) loses nothing.

Pattern 2 — Sliding window

Idea: keep a window `[start, end]` over a sequence; grow the end, shrink the start when a rule breaks. **Signal:** "longest/shortest/max-sum **contiguous** subarray or substring". **Cost:** O(n) — each index enters and leaves the window once.

```
function longestUnique(s) {
  const last = new Map();                 // char -> last index seen
  let start = 0, best = 0;
  for (let end = 0; end < s.length; end++) {
    const ch = s[end];
    if (last.has(ch) && last.get(ch) >= start) start = last.get(ch) + 1;   // shrink
    last.set(ch, end);
    best = Math.max(best, end - start + 1);
  }
  return best;
}
assert.equal(longestUnique('abcabcbb'), 3);

function maxSumK(nums, k) {               // fixed-size window: add the new, drop the old
  let sum = 0;
  for (let i = 0; i < k; i++) sum += nums[i];
  let best = sum;
  for (let i = k; i < nums.length; i++) { sum += nums[i] - nums[i - k]; best = Math.max(best, sum); }
  return best;
}
assert.equal(maxSumK([2, 1, 5, 1, 3, 2], 3), 9);
```

Pattern 3 — Hashing

Idea: trade memory for time — remember what you've seen in a `Map` / `Set` so a later lookup is O(1). **Signal:** "find a pair/duplicate", "group by", "count". **Cost:** O(n) time, O(n) space.

```
function twoSum(nums, target) {
  const indexOf = new Map();
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (indexOf.has(need)) return [indexOf.get(need), i];   // check BEFORE inserting
    indexOf.set(nums[i], i);
  }
  return null;
}
assert.deepEqual(twoSum([2, 7, 11, 15], 9), [0, 1]);
assert.deepEqual(twoSum([3, 3], 6), [0, 1]);

function groupAnagrams(words) {
  const groups = new Map();
  for (const w of words) {
    const key = [...w].sort().join('');      // canonical form = the hash key
    if (!groups.has(key)) groups.set(key, []);
    groups.get(key).push(w);
  }
  return [...groups.values()];
}
```

Checking before inserting is what makes `[3, 3]` work: the first `3` is not paired with itself. (`Map.groupBy` from §8.7 does the grouping step in one call.)

Pattern 4 — Stack (and monotonic stack)

Idea: a stack remembers "things still open" — brackets, unfinished elements. A **monotonic stack** keeps its contents increasing or decreasing and answers "next greater/smaller element" questions. **Cost:** O(n) — every index is pushed and popped at most once.

```
function isBalanced(s) {
  const pairs = { ')': '(', ']': '[', '}': '{' };
  const st = [];
  for (const ch of s) {
    if ('([{'.includes(ch)) st.push(ch);
    else if (ch in pairs) { if (st.pop() !== pairs[ch]) return false; }
  }
  return st.length === 0;                  // leftovers = unclosed brackets
}

function dailyTemperatures(t) {            // days until a warmer day
  const ans = new Array(t.length).fill(0);
  const st = [];                           // indices whose temperatures are decreasing
  for (let i = 0; i < t.length; i++) {
    while (st.length && t[i] > t[st.at(-1)]) { const j = st.pop(); ans[j] = i - j; }
    st.push(i);
  }
  return ans;
}
assert.deepEqual(dailyTemperatures([73, 74, 75, 71, 69, 72, 76, 73]), [1, 1, 4, 2, 1, 1, 0, 0]);
```

Pattern 5 — Prefix sums

Idea: precompute running totals `p[i] = nums[0] + … + nums[i-1]`; then any range sum is one subtraction. Combined with a hash map it counts subarrays with a given sum, even with negative numbers (where a sliding window fails). **Cost:** O(n) setup, O(1) per query.

```
function prefix(nums) {
  const p = [0];
  for (const x of nums) p.push(p.at(-1) + x);
  return (i, j) => p[j + 1] - p[i];        // sum of nums[i..j] inclusive
}
assert.equal(prefix([3, 1, 4, 1, 5, 9])(2, 5), 19);

function subarraySum(nums, k) {            // how many subarrays sum to k?
  const count = new Map([[0, 1]]);         // the empty prefix
  let sum = 0, total = 0;
  for (const x of nums) {
    sum += x;
    total += count.get(sum - k) ?? 0;      // earlier prefixes that leave exactly k
    count.set(sum, (count.get(sum) ?? 0) + 1);
  }
  return total;
}
assert.equal(subarraySum([1, -1, 0], 0), 3);
```

Pattern 6 — Intervals

Idea: sort by start, then sweep once, merging or counting overlaps. **Signal:** meetings, bookings, ranges. **Cost:** O(n log n) for the sort.

```
function mergeIntervals(intervals) {
  const sorted = intervals.toSorted((a, b) => a[0] - b[0]);   // don't mutate the input
  const out = [];
  for (const [s, e] of sorted) {
    const last = out.at(-1);
    if (last && s <= last[1]) last[1] = Math.max(last[1], e);   // overlap: extend
    else out.push([s, e]);
  }
  return out;
}
assert.deepEqual(mergeIntervals([[8, 10], [1, 3], [2, 6], [15, 18]]), [[1, 6], [8, 10], [15, 18]]);

function minRooms(meetings) {              // sweep line: sorted starts vs sorted ends
  const starts = meetings.map(m => m[0]).sort((a, b) => a - b);
  const ends = meetings.map(m => m[1]).sort((a, b) => a - b);
  let rooms = 0, best = 0, j = 0;
  for (const s of starts) {
    while (ends[j] <= s) { j++; rooms--; }  // a meeting ending at s frees its room
    rooms++; best = Math.max(best, rooms);
  }
  return best;
}
assert.equal(minRooms([[0, 30], [5, 10], [15, 20]]), 2);
```

Output of the array-patterns program:

```
pairSum [ 1, 4 ] | longestUnique("abcabcbb") 3
twoSum [ 0, 1 ] | groupAnagrams [["eat","tea","ate"],["tan","nat"],["bat"]]
dailyTemperatures 1,1,4,2,1,1,0,0
subarraySum([1,1,1],2) 2 | merge [[1,6],[8,10],[15,18]] | rooms 2
all array-pattern asserts passed
```

Pattern 7 — Heap (write your own `MinHeap`)

What it is: a **binary heap** is a complete binary tree stored in an array where every parent is ≤ its children (a *min*-heap). The smallest item is always at index 0. For index `i`, the children are `2i+1` and `2i+2`, the

parent is `(i-1)>>1`. **Signal:** "k largest/smallest", "merge k sorted lists", "median of a stream", "always process the cheapest next" (Dijkstra, scheduling). **Cost:** `push` / `pop` O(log n), `peek` O(1).

JavaScript has **no built-in priority queue**, so interviewers expect you to write one (or at least describe it). Twenty-five lines are enough:

```
export class MinHeap {
  #a = [];
  #less;
  constructor(less = (x, y) => x < y) { this.#less = less; }   // custom ordering
  get size() { return this.#a.length; }
  peek() { return this.#a[0]; }
  push(x) {                                    // add at the end, sift UP
    const a = this.#a; a.push(x);
    let i = a.length - 1;
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (!this.#less(a[i], a[p])) break;
      [a[i], a[p]] = [a[p], a[i]]; i = p;
    }
  }
  pop() {                                      // move the last to the root, sift DOWN
    const a = this.#a;
    if (a.length === 0) return undefined;
    const top = a[0], last = a.pop();
    if (a.length) {
      a[0] = last;
      let i = 0;
      for (;;) {
        const l = 2 * i + 1, r = l + 1;
        let m = i;
        if (l < a.length && this.#less(a[l], a[m])) m = l;
        if (r < a.length && this.#less(a[r], a[m])) m = r;
        if (m === i) break;
        [a[i], a[m]] = [a[m], a[i]]; i = m;
      }
    }
    return top;
  }
}

function kLargest(nums, k) {                   // min-heap of size k: O(n log k)
  const heap = new MinHeap();
  for (const x of nums) { heap.push(x); if (heap.size > k) heap.pop(); }
  const out = []; while (heap.size) out.push(heap.pop());
  return out.reverse();
}
assert.deepEqual(kLargest([3, 2, 1, 5, 6, 4], 2), [6, 5]);
```

The `less` parameter makes it a general **priority queue**: pass `(x, y) => x > y` for a max-heap, or compare a field. Dijkstra's shortest paths uses `[distance, node]` pairs:

```
function dijkstra(graph, src) {
  const dist = new Map([[src, 0]]);
  const pq = new MinHeap((x, y) => x[0] < y[0]);   // [distance, node]
  pq.push([0, src]);
  while (pq.size) {
    const [d, u] = pq.pop();
    if (d > dist.get(u)) continue;                 // stale entry: a shorter path was found
    for (const [v, w] of graph[u] ?? []) {
      const nd = d + w;
      if (nd < (dist.get(v) ?? Infinity)) { dist.set(v, nd); pq.push([nd, v]); }
    }
  }
  return Object.fromEntries(dist);
}
const roads = { A: [['B', 4], ['C', 1]], C: [['B', 2], ['D', 7]], B: [['D', 1]] };
assert.deepEqual(dijkstra(roads, 'A'), { A: 0, C: 1, B: 3, D: 4 });
```

Two heaps: the running median. To report the median of a stream after every new number, keep the smaller half in a **max-heap** (`low`) and the larger half in a **min-heap** (`high`), with `low` holding the same number of items or one more. The median is then `low`'s top, or the average of both tops. Each `add` is O(log n); `median` is O(1).

```
class MedianFinder {
  #low = new MinHeap((x, y) => x > y);        // max-heap: the smaller half
  #high = new MinHeap();                      // min-heap: the larger half
  add(x) {
    if (this.#low.size === 0 || x <= this.#low.peek()) this.#low.push(x); else this.#high.push(x);
    if (this.#low.size > this.#high.size + 1) this.#high.push(this.#low.pop());   // rebalance
    else if (this.#high.size > this.#low.size) this.#low.push(this.#high.pop());
  }
  median() {
    return this.#low.size > this.#high.size ? this.#low.peek() : (this.#low.peek() + this.#high.peek()) / 2;
  }
}
const mf = new MedianFinder();
assert.deepEqual([5, 15, 1, 3].map(x => { mf.add(x); return mf.median(); }), [5, 10, 5, 4]);
```

k-way merge. To merge k sorted lists, keep one "front" item per list in a min-heap; pop the smallest, then push the next item from the same list. With N items in total the cost is O(N log k) — better than concatenating and sorting (O(N log N)) when k is small.

```
function mergeKSorted(lists) {
  const pq = new MinHeap((x, y) => x[0] < y[0]);    // [value, list index, position]
  lists.forEach((l, i) => { if (l.length) pq.push([l[0], i, 0]); });
  const out = [];
  while (pq.size) {
    const [v, i, j] = pq.pop();
    out.push(v);
    if (j + 1 < lists[i].length) pq.push([lists[i][j + 1], i, j + 1]);   // refill from the same list
  }
  return out;
}
assert.deepEqual(mergeKSorted([[1, 4, 7], [2, 5, 8], [0, 3, 6, 9], []]), [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]);
```

Shortcut worth knowing: if you only need to sort once, `toSorted` is simpler and also O(n log n). A heap wins when items arrive over time or you need only the top k.

Pattern 8 — Binary search

Idea: halve the search space each step. **Signal:** sorted data, or a yes/no condition that flips once as a number grows ("the smallest speed that works"). **Cost:** O(log n) steps.

```
function lowerBound(a, target) {            // first index with a[i] >= target
  let lo = 0, hi = a.length;                // half-open range [lo, hi)
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (a[mid] < target) lo = mid + 1; else hi = mid;
  }
  return lo;                                 // a.length if every value is smaller
}
assert.equal(lowerBound([1, 3, 3, 5, 8], 3), 1);
assert.equal(lowerBound([1, 3, 3, 5, 8], 4), 3);

function minEatingSpeed(piles, hours) {     // binary search on the ANSWER
  let lo = 1, hi = Math.max(...piles);
  const ok = (k) => piles.reduce((t, p) => t + Math.ceil(p / k), 0) <= hours;
  while (lo < hi) { const mid = (lo + hi) >> 1; if (ok(mid)) hi = mid; else lo = mid + 1; }
  return lo;
}
assert.equal(minEatingSpeed([3, 6, 7, 11], 8), 4);
```

Use one template (half-open `[lo, hi)`, `lo < hi`, `hi = mid`) and you avoid the off-by-one bugs that make binary search famous. JavaScript has no `bisect` built-in; `indexOf` / `findIndex` are linear.

Pattern 9 — Breadth-first search (BFS)

Idea: explore in rings — all nodes 1 step away, then 2 steps, … using a queue. The first time you reach a node is by a **shortest path** (in steps). **Signal:** "minimum number of moves", unweighted shortest path, level-order traversal. **Cost:** O(V + E) (for a grid, O(rows × cols)).

```
function shortestPath(grid, [sr, sc], [tr, tc]) {
  const R = grid.length, C = grid[0].length;
  const dist = Array.from({ length: R }, () => new Array(C).fill(-1));   // -1 = unvisited
  const queue = [[sr, sc]]; dist[sr][sc] = 0;
  for (let head = 0; head < queue.length; head++) {       // index instead of shift()
    const [r, c] = queue[head];
    if (r === tr && c === tc) return dist[r][c];
    for (const [dr, dc] of [[1, 0], [-1, 0], [0, 1], [0, -1]]) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] !== '#' && dist[nr][nc] === -1) {
        dist[nr][nc] = dist[r][c] + 1; queue.push([nr, nc]);   // mark when ENQUEUED
      }
    }
  }
  return -1;
}
assert.equal(shortestPath(['..#.', '..#.', '....'], [0, 0], [0, 3]), 7);
```

Mark a cell visited when you **enqueue** it, not when you dequeue it — otherwise the same cell enters the queue many times.

Pattern 10 — Depth-first search (DFS)

Idea: go as deep as possible, then back up. **Signal:** connectivity ("number of islands"), cycle detection, topological order, exploring every path. **Cost:** O(V + E).

```
function numIslands(rows) {
  const g = rows.map(r => [...r]);          // copy: don't destroy the caller's grid
  let count = 0;
  for (let r = 0; r < g.length; r++) for (let c = 0; c < g[0].length; c++) {
    if (g[r][c] !== '1') continue;
    count++;
    const st = [[r, c]]; g[r][c] = '0';     // explicit stack = iterative DFS
    while (st.length) {
      const [x, y] = st.pop();
      for (const [nx, ny] of [[x + 1, y], [x - 1, y], [x, y + 1], [x, y - 1]]) {
        if (g[nx]?.[ny] === '1') { g[nx][ny] = '0'; st.push([nx, ny]); }   // ?. handles edges
      }
    }
  }
  return count;
}
assert.equal(numIslands(['11000', '11000', '00100', '00011']), 3);

function topoSort(n, edges) {               // recursive DFS with three colours
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) adj[u].push(v);
  const state = new Array(n).fill(0);       // 0 new, 1 on the current path, 2 done
  const order = [];
  const visit = (u) => {
    if (state[u] === 1) throw new Error('cycle');
    if (state[u] === 2) return;
    state[u] = 1;
    for (const v of adj[u]) visit(v);
    state[u] = 2; order.push(u);            // post-order
  };
  for (let u = 0; u < n; u++) visit(u);
  return order.reverse();
}
assert.deepEqual(topoSort(4, [[0, 1], [0, 2], [1, 3], [2, 3]]), [0, 2, 1, 3]);
```

JavaScript-specific: recursion depth is limited (roughly ten thousand frames in V8, depending on frame size), and there's no tail-call optimisation in V8. For inputs that can be deep — a 1000×1000 grid of land — use the iterative version with an explicit stack.

Pattern 11 — Dynamic programming with memoization

What it is: **dynamic programming (DP)** solves a problem by combining answers to smaller versions of the same problem, storing each answer so it is computed once. **Top-down** = recursion + a cache (**memoization**); **bottom-up** =

fill a table in order. **Signal:** "count the ways", "minimum cost", "longest …", when the plain recursion repeats the same calls. **Cost:** (number of states) × (work per state).

```
function climb(n, memo = new Map()) {       // ways to climb n stairs taking 1 or 2 steps
  if (n <= 1) return 1;
  if (memo.has(n)) return memo.get(n);
  const v = climb(n - 1, memo) + climb(n - 2, memo);
  memo.set(n, v);
  return v;
}
assert.equal(climb(70), 308061521170129);   // exact: below 2**53
assert.equal(Number.isSafeInteger(climb(80)), false);   // too big: use BigInt

function coinChange(coins, amount) {        // bottom-up: fewest coins for each amount
  const dp = new Array(amount + 1).fill(Infinity); dp[0] = 0;
  for (let a = 1; a <= amount; a++)
    for (const c of coins) if (c <= a) dp[a] = Math.min(dp[a], dp[a - c] + 1);
  return dp[amount] === Infinity ? -1 : dp[amount];
}
assert.equal(coinChange([1, 2, 5], 11), 3);

function lcs(a, b) {                        // longest common subsequence: 2D table
  const dp = Array.from({ length: a.length + 1 }, () => new Array(b.length + 1).fill(0));
  for (let i = 1; i <= a.length; i++) for (let j = 1; j <= b.length; j++)
    dp[i][j] = a[i - 1] === b[j - 1] ? dp[i - 1][j - 1] + 1 : Math.max(dp[i - 1][j], dp[i][j - 1]);
  return dp[a.length][b.length];
}
assert.equal(lcs('abcde', 'ace'), 3);
```

Without the memo, `climb(70)` would make about 6×10^{14} calls; with it, about 140. Note the JavaScript twist: the answer for 80 stairs is larger than `2**53`, so a real solution would switch to `BigInt` (§17.1). For multi-argument states, use a string key (`` `${i},${j}` ``) or a 2D array.

Pattern 12 — Backtracking

Idea: build a solution one choice at a time; after exploring a choice, **undo** it and try the next (choose → explore → un-choose). Prune branches that can't succeed. **Signal:** "list all subsets/permutations/combinations", puzzles (N-Queens, Sudoku). **Cost:** exponential — 2^n subsets, $n!$ permutations — so inputs are small.

```
function subsets(nums) {
  const out = [], cur = [];
  const go = (i) => {
    if (i === nums.length) { out.push([...cur]); return; }   // COPY the current path
    cur.push(nums[i]); go(i + 1); cur.pop();                 // choose, explore, un-choose
    go(i + 1);                                               // skip nums[i]
  };
  go(0);
  return out;
}
assert.deepEqual(subsets([1, 2]), [[1, 2], [1], [2], []]);

function nQueens(n) {                        // pruning with Sets of attacked lines
  const cols = new Set(), d1 = new Set(), d2 = new Set();
  let count = 0;
  const place = (r) => {
    if (r === n) { count++; return; }
    for (let c = 0; c < n; c++) {
      if (cols.has(c) || d1.has(r - c) || d2.has(r + c)) continue;
      cols.add(c); d1.add(r - c); d2.add(r + c);
      place(r + 1);
      cols.delete(c); d1.delete(r - c); d2.delete(r + c);
    }
  };
  place(0);
  return count;
}
assert.equal(nQueens(8), 92);
```

The most common JavaScript bug here is `out.push(cur)` without copying: every entry would be the *same* array, which ends up empty. (`permutations` with a `used` array is in the code file.)

Output of the search/graph/DP program:

```
heap drain 1,2,3,5,8,9 | kLargest [ 6, 5 ] | dijkstra {"A":0,"B":3,"C":1,"D":4}
running medians 5,10,5,4 | k-way merge 0,1,2,3,4,5,6,7,8,9
lowerBound 4 -> 3 | minEatingSpeed 4
maze steps 7 | islands 3 | topo 0,2,1,3
climb(70) 308061521170129 | climb(80) safe? false | coinChange 11 3 | lcs 3
subsets [[1,2],[1],[2],[]] | perms of 3: 6 | 8-queens 92
all search/graph/DP asserts passed
```

Pattern 13 — Linked lists

What it is: a **linked list** is a chain of nodes; each node holds a value and a `next` reference to the following node (`null` at the end). Unlike an array there is no index: reaching the k-th node means walking k steps, but inserting or removing a node you already hold is O(1) — you only rewire references. JavaScript has

no built-in linked list, so problems hand you a `{ val, next }` shape. **Signal:** "reverse", "remove the k-th from the end", "merge", "where do two lists meet". **Cost:** usually O(n) time, O(1) extra space.

```
class ListNode { constructor(val, next = null) { this.val = val; this.next = next; } }

function reverseList(head) {                // flip every arrow while walking
  let prev = null, cur = head;
  while (cur) {
    const next = cur.next;                  // save before overwriting
    cur.next = prev;
    prev = cur; cur = next;
  }
  return prev;                              // the old tail is the new head
}
assert.deepEqual(toArray(reverseList(fromArray([1, 2, 3, 4]))), [4, 3, 2, 1]);

function removeKthFromEnd(head, k) {        // one pass: `lead` runs k nodes ahead
  const dummy = new ListNode(0, head);      // dummy head: removing the head needs no special case
  let lead = dummy, trail = dummy;
  for (let i = 0; i < k; i++) lead = lead.next;
  while (lead.next) { lead = lead.next; trail = trail.next; }
  trail.next = trail.next.next;             // trail sits just before the node to remove
  return dummy.next;
}
assert.deepEqual(toArray(removeKthFromEnd(fromArray([1, 2, 3, 4, 5]), 2)), [1, 2, 3, 5]);

function intersection(a, b) {               // first node shared by two lists, or null
  let p = a, q = b;
  while (p !== q) { p = p ? p.next : b; q = q ? q.next : a; }
  return p;
}
```

Why `intersection` works: pointer `p` walks list A then list B, `q` walks B then A. Both travel `lenA + lenB` steps, so they line up and reach the shared node at the same moment — or both reach `null` together if there is none. It compares **nodes** with `!==`, not values: two lists that merely contain equal numbers do not intersect. The **dummy head** trick (a fake node before the real head) appears in almost every list problem because it removes the "what if I change the head?" branch. `fromArray` / `toArray` helpers are in the code file.

Pattern 14 — Fast & slow pointers

Idea: two pointers move through a sequence at different speeds — slow one step, fast two steps. If the sequence loops, fast eventually laps slow and they meet; if it ends, fast reaches the end when slow is halfway. **Signal:** "is there a cycle / where does it start", "middle of a list", any "does repeating this step loop forever?" question. **Cost:** O(n) time, **O(1) space** — the point, since a `Set` of visited nodes would also work but costs O(n).

```
function cycleStart(head) {                  // Floyd's algorithm
  let slow = head, fast = head;
  while (fast && fast.next) {
    slow = slow.next; fast = fast.next.next;
    if (slow === fast) {                     // met inside the loop
      let p = head;
      while (p !== slow) { p = p.next; slow = slow.next; }
      return p;                              // they meet again at the loop's first node
    }
  }
  return null;                               // fast fell off the end: no cycle
}

function middle(head) {                      // second middle for even lengths
  let slow = head, fast = head;
  while (fast && fast.next) { slow = slow.next; fast = fast.next.next; }
  return slow;
}
assert.equal(middle(fromArray([1, 2, 3, 4, 5])).val, 3);

function isHappy(n) {                        // n -> sum of squared digits; reaches 1 or loops
  const next = (x) => { let s = 0; while (x > 0) { const d = x % 10; s += d * d; x = Math.floor(x / 10); }
return s; };
  let slow = n, fast = next(n);
  while (fast !== 1 && slow !== fast) { slow = next(slow); fast = next(next(fast)); }
  return fast === 1;
}
assert.equal(isHappy(19), true);             // 19 -> 82 -> 68 -> 100 -> 1
assert.equal(isHappy(2), false);
```

Why the second phase finds the loop's start: say the start is `a` steps from the head and the pointers meet `b` steps into a loop of length `c`. Fast walked twice as far, so `2(a + b) = a + b + k·c` for some whole number `k`, which gives `a = k·c − b`. Walking `a` more steps from the meeting point therefore lands exactly on the loop's start — the same place a pointer starting at the head reaches after `a` steps. `isHappy` shows the pattern without any list: the "next" function defines an implicit chain.

Pattern 15 — Trees

What it is: a **binary tree** is a set of nodes where each node has at most two children (`left`, `right`); the top node is the **root**, nodes without children are **leaves**. A **binary search tree (BST)** adds a rule: every value in the left subtree is smaller than the node, every value in the right subtree larger. Most tree answers are a short recursive function that asks each subtree a question and combines the two answers. **Signal:** "validate",

"lowest common ancestor", "path sum", "rebuild / encode a tree". **Cost:** O(n) time, O(h) stack for a tree of height `h` (h = log n when balanced, n when it degenerates into a line).

```
class TreeNode { constructor(val, left = null, right = null) { this.val = val; this.left = left; this.right = right; } }

function isValidBST(node, lo = -Infinity, hi = Infinity) {   // pass the allowed range DOWN
  if (!node) return true;
  if (node.val <= lo || node.val >= hi) return false;
  return isValidBST(node.left, lo, node.val) && isValidBST(node.right, node.val, hi);
}
assert.equal(isValidBST(fromLevels([5, 1, 7, null, null, 4, 8])), false);   // 4 is right of 5

function lca(root, p, q) {                  // lowest common ancestor of values p and q
  if (!root || root.val === p || root.val === q) return root;
  const left = lca(root.left, p, q), right = lca(root.right, p, q);
  return left && right ? root : left ?? right;   // found on both sides: this node splits them
}
const family = fromLevels([3, 5, 1, 6, 2, 0, 8, null, null, 7, 4]);
assert.equal(lca(family, 7, 4).val, 2);

function serialize(root) {                  // preorder, '#' marks an empty child
  const out = [];
  const go = (n) => { if (!n) { out.push('#'); return; } out.push(n.val); go(n.left); go(n.right); };
  go(root);
  return out.join(',');
}
function deserialize(text) {
  const tokens = text.split(',');
  let i = 0;
  const go = () => {
    const t = tokens[i++];
    if (t === '#') return null;
    const node = new TreeNode(Number(t));
    node.left = go(); node.right = go();      // same order as serialize
    return node;
  };
  return go();
}
assert.equal(serialize(fromLevels([1, 2, 3, null, null, 4, 5])), '1,2,#,#,3,4,#,#,5,#,#');
```

The classic BST bug is comparing a node only with its parent: in `[5, 1, 7, null, null, 4, 8]`, 4 is a correct left child of 7 but sits in 5's *right* subtree, so the tree is invalid. Passing the range `(lo, hi)` down catches it. Serialization works because preorder plus explicit `#` markers fixes the shape: the reader always knows whether the next token is a left child, a right child or an empty slot. (`fromLevels`, which builds a tree from a level-order array with `null` gaps, is in the code file; deep, unbalanced trees hit the recursion-depth limit from Pattern 10 — switch to an explicit stack.)

Pattern 16 — Tries

What it is: a **trie** (prefix tree, pronounced "try") stores strings one character per level; words that share a prefix share the same path from the root, and a flag marks "a word ends here". Looking up a word or prefix

of length `L` costs O(L), no matter how many words are stored. **Signal:** "autocomplete", "starts with", "search with a wildcard", "find many words in a grid". **Cost:** O(L) per insert/search; memory O(total characters).

```
class Trie {
  #root = { next: new Map(), end: false };
  insert(word) {
    let n = this.#root;
    for (const ch of word) {
      if (!n.next.has(ch)) n.next.set(ch, { next: new Map(), end: false });
      n = n.next.get(ch);
    }
    n.end = true;                          // "a word finishes here"
  }
  #walk(prefix) {
    let n = this.#root;
    for (const ch of prefix) { n = n.next.get(ch); if (!n) return null; }
    return n;
  }
  search(word) { return this.#walk(word)?.end === true; }
  startsWith(prefix) { return this.#walk(prefix) !== null; }
  match(pattern) {                         // '.' matches any one letter
    const go = (n, i) => {
      if (i === pattern.length) return n.end;
      if (pattern[i] !== '.') { const nx = n.next.get(pattern[i]); return nx ? go(nx, i + 1) : false; }
      for (const nx of n.next.values()) if (go(nx, i + 1)) return true;   // try every branch
      return false;
    };
    return go(this.#root, 0);
  }
}
// words: car, card, care, cat, dog
assert.equal(trie.search('ca'), false);      // a prefix, not a stored word
assert.equal(trie.match('c.r.'), true);      // card, care
```

The third classic, **word search on a board** (find every listed word that can be traced through neighbouring cells, each cell used once per word), combines a trie with Pattern 12's backtracking: one DFS from each cell walks the board and the trie together, and stops as soon as the letters so far are not a prefix of any word. That prune is what makes it fast compared with searching for each word separately.

```
function findWords(board, words) {
  const root = {};
  for (const w of words) { let n = root; for (const ch of w) n = n[ch] ??= {}; n.$word = w; }
  const grid = board.map(row => [...row]);   // copy: cells are marked while exploring
  const R = grid.length, C = grid[0].length, found = [];
  const dfs = (r, c, node) => {
    const ch = grid[r][c], next = node[ch];
    if (!next) return;                       // no word continues this way: prune
    if (next.$word) { found.push(next.$word); delete next.$word; }   // report once
    grid[r][c] = '#';                        // choose
    for (const [dr, dc] of [[1, 0], [-1, 0], [0, 1], [0, -1]]) {
      const nr = r + dr, nc = c + dc;
      if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] !== '#') dfs(nr, nc, next);
    }
    grid[r][c] = ch;                         // un-choose
  };
  for (let r = 0; r < R; r++) for (let c = 0; c < C; c++) dfs(r, c, root);
  return found.sort();
}
assert.deepEqual(findWords(['cats', 'ored', 'waxy'], ['cat', 'cats', 'car', 'core', 'cow', 'dog', 'red',
'wax', 'tax']),
  ['car', 'cat', 'cats', 'core', 'cow', 'red', 'wax']);
```

JavaScript notes: a `Map` per node is the clean choice; a plain object (`n[ch] ??= {}`) is shorter, but then a stored key such as `$word` must never collide with a letter. For lowercase-only input, a 26-slot array per node

is the fastest variant (§17.1).

Output of the list/tree/trie program:

```
reverse 4,3,2,1 | remove 2nd from end 1,2,3,5 | intersection at 8
cycle starts at 3 | middle of 1..4 3 | happy 19? true | happy 2? false
validBST true false | lca(7,4) 2 | serialized 1,2,#,#,3,4,#,#,5,#,#
trie search car true | prefix ca true | match c.r. true | board words car,cat,cats,core,cow,red,wax
all list/tree/trie asserts passed
```

Pattern 17 — Topological sort and union-find

What it is: two graph tools that plain BFS/DFS (Patterns 9–10) don't cover directly. A **topological order** lists the nodes of a directed graph so every edge goes from earlier to later ("take prerequisites first"); it exists only if the graph has no cycle. **Kahn's algorithm** builds it with BFS: repeatedly take a node whose **in-degree** (number of incoming edges still unprocessed) is 0. A **union-find** (disjoint-set union) keeps a collection of groups and answers "are x and y in the same group?" and "merge their groups" in almost constant time.
Signal: "order of tasks with dependencies", "can all courses be finished" → topological sort; "number of connected groups", "does this edge create a cycle", merging accounts or friend circles → union-find. **Cost:**

Kahn O(V + E); union-find O(α(n)) per operation, where α (the inverse Ackermann function) is below 5 for any real input.

```
function courseOrder(n, prereqs) {           // prereqs: [course, mustTakeFirst]
  const adj = Array.from({ length: n }, () => []);
  const indeg = new Array(n).fill(0);
  for (const [c, p] of prereqs) { adj[p].push(c); indeg[c]++; }
  const order = [];
  for (let i = 0; i < n; i++) if (indeg[i] === 0) order.push(i);
  for (let h = 0; h < order.length; h++)     // `order` doubles as the queue
    for (const v of adj[order[h]]) if (--indeg[v] === 0) order.push(v);
  return order.length === n ? order : null;  // null = cycle: impossible schedule
}
assert.deepEqual(courseOrder(4, [[1, 0], [2, 0], [3, 1], [3, 2]]), [0, 1, 2, 3]);
assert.equal(courseOrder(2, [[0, 1], [1, 0]]), null);

class UnionFind {
  constructor(n) {
    this.parent = Array.from({ length: n }, (_, i) => i);   // everyone starts alone
    this.size = new Array(n).fill(1);
    this.sets = n;
  }
  find(x) {                                  // follow parents to the group's root
    while (this.parent[x] !== x) {
      this.parent[x] = this.parent[this.parent[x]];        // path halving: flatten as we go
      x = this.parent[x];
    }
    return x;
  }
  union(a, b) {
    let ra = this.find(a), rb = this.find(b);
    if (ra === rb) return false;             // already in the same group
    if (this.size[ra] < this.size[rb]) [ra, rb] = [rb, ra];
    this.parent[rb] = ra;                    // hang the smaller tree under the bigger one
    this.size[ra] += this.size[rb];
    this.sets--;
    return true;
  }
}
const countComponents = (n, edges) => { const uf = new UnionFind(n); for (const [a, b] of edges) uf.union(a,
b); return uf.sets; };
assert.equal(countComponents(5, [[0, 1], [1, 2], [3, 4]]), 2);

function redundantEdge(edges) {              // first edge that closes a cycle (nodes 1..n)
  const uf = new UnionFind(edges.length + 1);
  for (const e of edges) if (!uf.union(e[0], e[1])) return e;
  return null;
}
assert.deepEqual(redundantEdge([[1, 2], [2, 3], [3, 4], [1, 4], [1, 5]]), [1, 4]);
```

Kahn vs the DFS topological sort in Pattern 10: both are O(V + E). Kahn detects a cycle by counting ("fewer than n nodes came out") instead of by recursion colours, needs no recursion (no depth limit), and processes the graph in "waves" — handy when asked for the minimum number of semesters. Union-find answers connectivity for edges that arrive one at a time, where rerunning BFS after each edge would cost O(V + E) each time. The two optimisations (path halving and union by size) are what make it fast; without them a chain of unions can make `find` O(n).

Pattern 18 — Greedy

What it is: a **greedy** algorithm makes the choice that looks best right now and never revisits it. It is fast (usually one pass, often after a sort) but only correct when you can argue that a locally best choice never blocks a globally best answer — so in the interview, always say *why* the greedy choice is safe, or give a

counterexample and switch to DP (Pattern 11). **Signal:** "minimum number of …", "can you reach …", scheduling, and problems where a sort plus one sweep feels natural. **Cost:** O(n) or O(n log n) with a sort.

```
function canJump(nums) {                    // nums[i] = max jump length from i
  let reach = 0;                            // farthest index reachable so far
  for (let i = 0; i < nums.length; i++) {
    if (i > reach) return false;            // stuck before i
    reach = Math.max(reach, i + nums[i]);
  }
  return true;
}
assert.equal(canJump([3, 2, 1, 0, 4]), false);

function gasStation(gas, cost) {            // start index for a full circle, or -1
  let total = 0, tank = 0, start = 0;
  for (let i = 0; i < gas.length; i++) {
    const d = gas[i] - cost[i];
    total += d; tank += d;
    if (tank < 0) { start = i + 1; tank = 0; }   // nobody in start..i can be the answer
  }
  return total >= 0 ? start : -1;
}
assert.equal(gasStation([1, 2, 3, 4, 5], [3, 4, 5, 1, 2]), 3);

function candies(ratings) {                 // everyone >= 1; higher rating than a neighbour => more
  const n = ratings.length, c = new Array(n).fill(1);
  for (let i = 1; i < n; i++) if (ratings[i] > ratings[i - 1]) c[i] = c[i - 1] + 1;
  for (let i = n - 2; i >= 0; i--) if (ratings[i] > ratings[i + 1]) c[i] = Math.max(c[i], c[i + 1] + 1);
  return c.reduce((a, b) => a + b, 0);
}
assert.equal(candies([1, 3, 4, 5, 2]), 11);  // 1, 2, 3, 4, 1
```

The safety arguments, in one line each. **Jump game:** every index up to `reach` is reachable, so extending `reach` from each of them loses nothing. **Gas station:** if the tank goes negative at `i` after starting at `s`, every station between `s` and `i` arrived with at least 0 fuel from `s`, so starting there is no better — skip to `i + 1`; and if total gas covers total cost, the last surviving start completes the loop. **Candies:** the left-to-right pass satisfies every "better than my left neighbour" rule with the fewest candies, the right-to-left pass does the same for right neighbours, and `Math.max` keeps both. When greedy fails — coin change with coins `[1, 3, 4]` and amount 6 (greedy picks 4+1+1, the best is 3+3) — use DP. (Pattern 6's meeting-rooms and merge-intervals are greedy too: sort, then sweep.)

Pattern 19 — Sort & search: quickselect, Dutch flag, merge sort

Idea: some questions are really about the sorting algorithms themselves: find the k-th item *without* fully sorting, partition in place, or sort stably. **Quickselect** partitions like quicksort (a **pivot** value; smaller items to its left, the rest to its right) but then continues into only the side that contains the answer. **Signal:** "k-th largest/smallest", "median", "sort 0/1/2 in one pass", "sort a linked list", "count inversions". **Cost:** quickselect O(n) average (O(n²) worst, made unlikely by a random pivot); Dutch flag O(n) one pass, O(1) space; merge sort O(n log n) always, O(n) extra space, **stable**.

```
function kthLargest(nums, k) {              // quickselect
  const a = [...nums], target = a.length - k;   // k-th largest = index n-k ascending
  let lo = 0, hi = a.length - 1;
  for (;;) {
    const r = lo + Math.floor(Math.random() * (hi - lo + 1));
    [a[r], a[hi]] = [a[hi], a[r]];          // random pivot, moved to the end
    const pivot = a[hi];
    let store = lo;                         // Lomuto partition
    for (let i = lo; i < hi; i++) if (a[i] < pivot) { [a[i], a[store]] = [a[store], a[i]]; store++; }
    [a[store], a[hi]] = [a[hi], a[store]];   // the pivot is now in its final sorted place
    if (store === target) return a[store];
    if (store < target) lo = store + 1; else hi = store - 1;   // keep ONE side
  }
}
assert.equal(kthLargest([3, 2, 1, 5, 6, 4], 2), 5);

function sortColors(a) {                    // Dutch national flag: 0s, 1s, 2s in one pass
  let lo = 0, mid = 0, hi = a.length - 1;    // [0,lo)=0s  [lo,mid)=1s  [mid,hi]=unknown  (hi,end]=2s
  while (mid <= hi) {
    if (a[mid] === 0) { [a[lo], a[mid]] = [a[mid], a[lo]]; lo++; mid++; }
    else if (a[mid] === 1) mid++;
    else { [a[mid], a[hi]] = [a[hi], a[mid]]; hi--; }   // don't move mid: new a[mid] is unchecked
  }
  return a;
}
assert.deepEqual(sortColors([2, 0, 2, 1, 1, 0]), [0, 0, 1, 1, 2, 2]);

function mergeSort(a, cmp = (x, y) => x - y) {
  if (a.length <= 1) return a;
  const mid = a.length >> 1;
  const L = mergeSort(a.slice(0, mid), cmp), R = mergeSort(a.slice(mid), cmp);
  const out = [];
  let i = 0, j = 0;
  while (i < L.length && j < R.length) out.push(cmp(L[i], R[j]) <= 0 ? L[i++] : R[j++]);   // <= : stable
  return out.concat(L.slice(i), R.slice(j));
}
const people = [['ann', 30], ['bob', 25], ['cy', 30], ['di', 25]];
assert.deepEqual(mergeSort(people, (x, y) => x[1] - y[1]).map(p => p[0]), ['bob', 'di', 'ann', 'cy']);
```

Quickselect vs a heap (Pattern 7): quickselect is O(n) on average but needs the whole array in memory and rearranges a copy; a size-k heap is O(n log k) but works on a stream. The code file also checks `kthLargest` against `toSorted` on 200 random arrays — a cheap way to test any randomised algorithm. Merge sort's merge step is the same idea as "merge two sorted lists" and, on a linked list, needs no extra array (find the middle with Pattern 14, sort both halves, merge). In everyday JavaScript, `toSorted` is already a stable O(n log n) sort (§17.1); write these only when the question is about the algorithm.

Pattern 20 — Bit manipulation

What it is: treating a number as its binary digits (**bits**) and using the bitwise operators: `&` (and), `|` (or), `^` (xor: 1 when the bits differ), `~` (not), `<<` (shift left = ×2), `>>` (shift right keeping the sign), `>>>` (shift right filling with 0). **Signal:** "without extra space, find the one that appears once", "count 1-bits", "power of two", subsets as bitmasks. **Cost:** O(1) per operation. **The JavaScript twist:** numbers are 64-bit

floats, but every bitwise operator first converts to a **32-bit signed integer**, so results can come out negative and values above 2^{32} lose their high bits; `x >>> 0` reinterprets the 32 bits as unsigned.

```
const singleNumber = (nums) => nums.reduce((acc, x) => acc ^ x, 0);   // x ^ x = 0, x ^ 0 = x
assert.equal(singleNumber([4, 1, 2, 1, 2]), 4);

function countBits(n) {                     // number of 1-bits for every i in 0..n
  const bits = new Array(n + 1).fill(0);
  for (let i = 1; i <= n; i++) bits[i] = bits[i >> 1] + (i & 1);   // DP: drop the last bit
  return bits;
}
assert.deepEqual(countBits(5), [0, 1, 1, 2, 1, 2]);

// swap every pair of neighbouring bits: 0xAAAAAAAA = odd positions, 0x55555555 = even
const swapOddEvenBits = (x) => (((x & 0xAAAAAAAA) >>> 1) | ((x & 0x55555555) << 1)) >>> 0;
assert.equal(swapOddEvenBits(0b10110), 0b101001);
assert.equal(swapOddEvenBits(0x40000000), 0x80000000);   // 2147483648, thanks to >>> 0
assert.equal(0x40000000 << 1, -2147483648);              // the trap without it

const isPowerOfTwo = (n) => n > 0 && (n & (n - 1)) === 0;   // n & (n-1) clears the lowest 1-bit
```

`singleNumber` works in any order because xor is commutative and associative: the pairs cancel and only the lone value is left, with O(1) space instead of a `Set`. `countBits` is a tiny DP: `i >> 1` is `i` without its last bit (already computed) and `i & 1` is that last bit. In the swap, the final `>>> 0` matters: shifting a 1 into bit 31 makes the signed result negative. Because the file asserts against a slow bit-by-bit version on edge values (`0`, `0x80000000`, `0xFFFFFFFF`), the sign bug would be caught. For numbers beyond 32 bits, `BigInt` supports the same operators (`5n & 3n`), except `>>>`.

Pattern 21 — Math & geometry

Idea: simulation and number facts rather than a data structure: walk a matrix in a given order, handle integer limits, compare slopes exactly. The skill is spotting the invariant and the edge cases. **Signal:** "spiral / rotate a matrix", "reverse digits … return 0 on overflow", "points on a line", gcd, modular arithmetic. **Cost:** problem-specific — here O(rows × cols), O(digits) and O(n^2).

```
function spiral(m) {                        // peel layers with four shrinking walls
  const out = [];
  let top = 0, bottom = m.length - 1, left = 0, right = m[0].length - 1;
  while (top <= bottom && left <= right) {
    for (let c = left; c <= right; c++) out.push(m[top][c]);
    top++;
    for (let r = top; r <= bottom; r++) out.push(m[r][right]);
    right--;
    if (top <= bottom) { for (let c = right; c >= left; c--) out.push(m[bottom][c]); bottom--; }
    if (left <= right) { for (let r = bottom; r >= top; r--) out.push(m[r][left]); left++; }
  }
  return out;
}
assert.deepEqual(spiral([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]), [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6,
7]);

function reverseInt(x) {                    // 0 if the result leaves the 32-bit signed range
  const sign = x < 0 ? -1 : 1;
  let n = Math.abs(x), r = 0;
  while (n > 0) { r = r * 10 + (n % 10); n = Math.floor(n / 10); }
  r *= sign;
  return r < -(2 ** 31) || r > 2 ** 31 - 1 ? 0 : r;
}
assert.equal(reverseInt(-120), -21);
assert.equal(reverseInt(1534236469), 0);     // 9646324351 > 2^31 - 1

function maxPointsOnLine(points) {          // most points on one straight line
  if (points.length <= 2) return points.length;
  const gcd = (a, b) => (b === 0 ? Math.abs(a) : gcd(b, a % b));
  let best = 0;
  for (let i = 0; i < points.length; i++) {
    const byDir = new Map();
    let same = 1, most = 0;                 // `same` = the anchor plus exact duplicates
    for (let j = i + 1; j < points.length; j++) {
      let dx = points[j][0] - points[i][0], dy = points[j][1] - points[i][1];
      if (dx === 0 && dy === 0) { same++; continue; }
      const g = gcd(dx, dy);
      dx /= g; dy /= g;
      if (dx < 0 || (dx === 0 && dy < 0)) { dx = -dx; dy = -dy; }   // one sign per direction
      const key = `${dx}/${dy}`;
      const c = (byDir.get(key) ?? 0) + 1;
      byDir.set(key, c);
      most = Math.max(most, c);
    }
    best = Math.max(best, most + same);
  }
  return best;
}
assert.equal(maxPointsOnLine([[1, 1], [3, 2], [5, 3], [4, 1], [2, 3], [1, 4]]), 4);
```

The two `if`s in `spiral` stop a single remaining row or column from being walked twice (try a 3×1 matrix). In `reverseInt`, JavaScript is *easier* than languages with fixed-size integers: numbers are exact up to 2^{53}, so the intermediate value cannot overflow and one range check at the end is enough; in Java or C++ you must check *before* `r * 10`. In `maxPointsOnLine`, a floating-point slope `dy / dx` would be both imprecise (in the code file, `94911150 / 94911151 === 94911151 / 94911152` is `true` although the two directions differ, so three non-collinear points would be counted as one line) and infinite for vertical lines; the reduced integer pair as a `Map` key avoids both — the same "canonical key" idea as Pattern 3's anagram grouping.

Output of the graph/greedy/sort/bit/math program:

```
course order 0,1,2,3 | cyclic null | components 2 | redundant [2,3]
canJump true false | gas start 3 | candies 5 11
2nd largest 5 | colors 0,0,1,1,2,2 | mergeSort stable bob,di,ann,cy
single 4 | countBits(5) 0,1,1,2,1,2 | swap bits 22 -> 41 | swap 0x40000000 -> 2147483648
spiral 1,2,3,6,9,8,7,4,5 | reverse -21 0 | max collinear 4
all graph/greedy/sort/bit/math asserts passed
```

Summary table

#	Pattern	Signal words	Time	Extra space
1	Two pointers	sorted, pair, palindrome, in place	O(n)	O(1)
2	Sliding window	longest/shortest contiguous	O(n)	O(k)
3	Hashing	seen before, count, group	O(n)	O(n)
4	Stack / monotonic stack	brackets, next greater	O(n)	O(n)
5	Prefix sums	range sum, subarray sum = k	O(n) + O(1)/query	O(n)
6	Intervals	overlap, merge, rooms	O(n log n)	O(n)
7	Heap (two heaps, k-way merge)	top k, cheapest next, running median	O(n log k)	O(k)
8	Binary search	sorted, minimum value that works	O(log n) × check	O(1)
9	BFS	fewest steps, levels	O(V + E)	O(V)
10	DFS	connected, cycle, order	O(V + E)	O(V)
11	DP + memo	count ways, min cost, overlapping calls	states × transitions	states
12	Backtracking	all combinations, puzzles	exponential	O(depth)
13	Linked lists	reverse, k-th from end, merge, intersection	O(n)	O(1)
14	Fast & slow pointers	cycle, cycle start, middle, "does it loop?"	O(n)	O(1)
15	Trees	validate BST, ancestor, path, serialize	O(n)	O(h)
16	Tries	prefix, autocomplete, wildcard, words in a grid	O(L) per word	O(total chars)
17	Topological sort / union-find	dependencies, order; groups, redundant edge	O(V + E); ≈ O(1) per op	O(V)
18	Greedy	reach, minimum count, schedule (prove the choice)	O(n) or O(n log n)	O(1)–O(n)
19	Sort & search	k-th largest, 3-way partition, stable sort	O(n) avg; O(n log n)	O(1); O(n)
20	Bit manipulation	appears once, count bits, power of two, masks	O(n) or O(1)	O(1)
21	Math & geometry	spiral, overflow, collinear points, gcd	problem-specific	O(1)–O(n)

In the interview

30-second answer — "How do you approach a coding problem?" Restate it; agree on input/output and edge cases (empty, duplicates, negatives). State the brute force and its cost. Find the pattern — sorted → binary search/two pointers; contiguous → window; lookups → hash map; fewest steps → BFS — and state the better complexity before coding. Then test by hand with a normal and an edge case.

- *Why BFS rather than DFS for shortest path?* BFS reaches nodes in order of distance, so the first visit is the shortest; DFS may find a long path first.

- *Top-down or bottom-up DP?* Top-down is easier to derive and computes only needed states; bottom-up avoids recursion-depth limits (important in JavaScript) and allows rolling arrays.
- *How do you implement a priority queue in JavaScript?* A binary heap over an array: push = append + sift up, pop = move last to root + sift down, both O(log n).
- *Why a dummy head in linked-list problems?* It gives every real node a predecessor, so removing or inserting at the head needs no special case.
- *How do you prove a greedy solution?* Show that the greedy choice can be swapped into any optimal answer without making it worse (an **exchange argument**), or that it keeps an invariant (jump game: everything up to `reach` is reachable). If you can't, try a small counterexample and fall back to DP.
- *Quickselect or heap for the k-th largest?* Quickselect: O(n) average, needs the whole array. Heap of size k: O(n log k), works on a stream and has no bad worst case.
- *Kahn's algorithm or DFS for topological order?* Both O(V + E). Kahn counts nodes to detect a cycle and needs no recursion; DFS uses three colours and post-order.

Self-check

1. Which pattern finds the length of the longest substring with at most two distinct letters? → Sliding window with a `Map` of counts.
2. Why does `subarraySum` start the map with `[0, 1]`? → So subarrays that start at index 0 (prefix sum exactly `k`) are counted.
3. What's wrong with marking BFS cells visited when you dequeue them? → Duplicates enter the queue; still correct but can blow up time and memory.
4. What's the parent index of heap node 6? → `(6 - 1) >> 1 = 2`.
5. In `removeKthFromEnd`, why start both pointers at a dummy node? → So that when k equals the list length, `trail` can stop *before* the head and remove it.
6. Why must `isValidBST` pass a range instead of comparing each node with its parent? → A node can be correct for its parent yet violate an ancestor further up (4 under 7 under 5's right side).
7. What does `union` return in `redundantEdge`, and why does `false` mean a cycle? → `false` = the two ends already had the same root, so a path between them existed and this edge closes a loop.
8. Why does `swapOddEvenBits` end with `>>> 0`? → Bitwise results are 32-bit **signed**; a 1 in bit 31 would print as a negative number. `>>> 0` reads the same bits as unsigned.

TL;DR

Recognise the pattern, state the complexity, then code. Two pointers/window/hashing/prefix sums are O(n) array tricks; stacks handle nesting; sort first for intervals; heaps (hand-written in JS) for top-k, Dijkstra, k-way merge and – two of them – a running median; binary search on data or on the answer; BFS for fewest steps, DFS (iterative for deep inputs) for connectivity; memoize overlapping recursion; backtrack with choose/explore/un-choose and copy results. Linked lists: rewire with a dummy head; fast & slow pointers find cycles and middles in O(1) space; trees: recursion that combines subtree answers (pass ranges down for BSTs); tries for prefixes; Kahn's algorithm for dependency order, union-find for growing groups; greedy only with a safety argument; quickselect for the k-th item, Dutch flag for 3-way partitions, merge sort when stability matters; bitwise ops are 32-bit signed (`>>> 0` for unsigned); in math problems use exact integer keys, not float slopes.

17.3 Implement-from-scratch classics

What it is

"Implement `X` without using the built-in" questions test whether you understand *how* everyday tools work: closures and timers (`debounce`, `throttle`), `this` and `apply` (`bind`), recursion and object types (deep clone, deep equal), `Map` ordering (LRU cache), and promises (`Promise.all`). Each implementation below is short, correct for the cases interviewers probe, and tested.

How the tests control time

`debounce` and `throttle` depend on time, and real timers make tests slow and flaky. `code/04_classics_functions.mjs` uses the **fake timers** from `node:test` (§16.5): `setTimeout` and `Date.now()` only

move forward when the test calls `tick(ms)`.

```
import { mock } from 'node:test';
mock.timers.enable({ apis: ['setTimeout', 'Date'] });
const tick = (ms) => mock.timers.tick(ms);
```

Debounce

What it is: *a* **debounced** function waits until calls *stop* for `wait` ms, then runs once with the latest arguments. Analogy: a lift door that stays open while people keep walking in, and closes only after nobody has come for a few seconds. **Use it for:** search-as-you-type, auto-save, resize handling.

```
function debounce(fn, wait) {
  let timer = null;
  function debounced(...args) {
    clearTimeout(timer);                          // every call restarts the countdown
    timer = setTimeout(() => { timer = null; fn.apply(this, args); }, wait);
  }
  debounced.cancel = () => { clearTimeout(timer); timer = null; };
  return debounced;
}

const calls = [];
const search = debounce((q) => calls.push(q), 300);
search('j'); tick(100); search('ja'); tick(100); search('jav'); // user still typing
tick(299); assert.deepEqual(calls, []);            // 299 ms after the last key: nothing yet
tick(1);   assert.deepEqual(calls, ['jav']);       // fires once, with the LAST args
search('x'); search.cancel(); tick(1000);
assert.deepEqual(calls, ['jav']);                  // cancelled
```

Output: `debounce calls: [ 'jav' ]`

Interviewers check: (1) the timer lives in a **closure** shared by all calls; (2) `debounced` is a regular `function` forwarding `this` and arguments with `apply`, so it works as a method; (3) a `cancel` method (Lodash also offers `flush` and `leading`).

Throttle

What it is: *a* **throttled** function runs **at most once per** `wait` **ms**, however often it's called. Analogy: a bus that leaves every 10 minutes whether 1 or 50 people arrive. **Use it for:** scroll and mousemove handlers, rate-

limited API calls. This version runs on the **leading** edge (immediately) and also on the **trailing** edge (once more at the end with the latest arguments) so the final position is never lost.

```
function throttle(fn, wait) {
  let last = -Infinity, timer = null, pendingArgs = null, pendingThis = null;
  function throttled(...args) {
    const now = Date.now(), remaining = wait - (now - last);
    if (remaining <= 0) {                          // window open: run now (leading edge)
      clearTimeout(timer); timer = null;
      last = now; fn.apply(this, args);
    } else {                                       // inside window: remember the latest call
      pendingArgs = args; pendingThis = this;
      timer ??= setTimeout(() => {                 // trailing edge
        last = Date.now(); timer = null;
        fn.apply(pendingThis, pendingArgs);
      }, remaining);
    }
  }
  return throttled;
}

const calls = [], t0 = Date.now();
const onScroll = throttle((y) => calls.push(`${y}@${Date.now() - t0}`), 100);
for (let t = 0; t <= 250; t += 10) { onScroll(t); tick(10); }  // an event every 10 ms
for (let i = 0; i < 20; i++) tick(10);                          // then 200 quiet ms
assert.deepEqual(calls, ['0@0', '90@100', '190@200', '250@300']);
```

Output: `throttle calls: [ '0@0', '90@100', '190@200', '250@300' ]` – 26 events became 4 calls, exactly 100 ms apart, and the last one carries the final value `250`.

	Debounce	Throttle
Runs	Once, after the calls stop	Regularly, at most once per interval
During a long burst	Never (until it ends)	Every `wait` ms
Typical use	Search box, auto-save, validation	Scroll, drag, resize-driven layout, analytics

In the browser, visual updates tied to scrolling are often better done with `requestAnimationFrame` (once per frame) – see §14.5.

Curry

What it is: **currying** turns `f(a, b, c)` into a chain `f(a)(b)(c)`. The usual interview version also accepts several arguments at once. It relies on `fn.length` – the number of declared parameters (it doesn't count default or rest parameters).

```
function curry(fn) {
  return function curried(...args) {
    if (args.length >= fn.length) return fn.apply(this, args);     // enough: call it
    return (...more) => curried.apply(this, [...args, ...more]);   // else collect more
  };
}
const add3 = curry((a, b, c) => a + b + c);
assert.equal(add3(1)(2)(3), 6);
assert.equal(add3(1, 2)(3), 6);
const add1 = add3(1);                                  // partials are reusable
assert.equal(add1(10)(20), 31); assert.equal(add1(0)(0), 1);

// Variant: unknown number of arguments, ended by an empty call
const sum = (a) => (b) => (b === undefined ? a : sum(a + b));
assert.equal(sum(1)(2)(3)(), 6);
```

Reusability works because each call builds a **new** argument array instead of pushing into a shared one — a common bug.

Compose and pipe

Function composition chains functions so the output of one is the input of the next. `compose(f, g)(x)` = `f(g(x))` (right to left, as in maths); `pipe(f, g)(x)` = `g(f(x))` (left to right, as you read).

```
const compose = (...fns) => (x) => fns.reduceRight((acc, f) => f(acc), x);
const pipe = (...fns) => (x) => fns.reduce((acc, f) => f(acc), x);
const inc = (x) => x + 1, dbl = (x) => x * 2;
assert.equal(compose(inc, dbl)(5), 11);               // inc(dbl(5))
assert.equal(pipe(inc, dbl)(5), 12);                  // dbl(inc(5))
assert.equal(pipe()(7), 7);                           // no functions = identity
```

Memoize

Memoization caches a pure function's results by its arguments. Key design questions: how to build the key (here `JSON.stringify(args)`, overridable), and where the cache lives (a `Map`, which can grow forever — production code bounds it, e.g. with the LRU cache below).

```
function memoize(fn, keyOf = (...args) => JSON.stringify(args)) {
  const cache = new Map();
  return function (...args) {
    const key = keyOf(...args);
    if (cache.has(key)) return cache.get(key);        // has(): a cached undefined still counts
    const value = fn.apply(this, args);
    cache.set(key, value);
    return value;
  };
}
let runs = 0;
const slowSquare = memoize((n) => { runs++; return n * n; });
slowSquare(4); slowSquare(4); slowSquare(5);
assert.equal(runs, 2);
// Recursive functions must call the MEMOIZED version, or only the top call is cached
const fib = memoize((n) => (n < 2 ? n : fib(n - 1) + fib(n - 2)));
```

Output: `memoize runs: 2 | fib(50): 12586269025`. `JSON.stringify` keys can't tell apart some values (`undefined` and `null` in arrays both become `null`; objects with different key orders differ) — mention it and offer a custom `keyOf`, or a `WeakMap` for single-object arguments.

Flatten

Flatten a nested array to a given depth, exactly like the built-in `arr.flat(depth)` (§7.5):

```
function flatten(arr, depth = Infinity) {
  const out = [];
  for (const x of arr) {
    if (Array.isArray(x) && depth > 0) out.push(...flatten(x, depth - 1));
    else out.push(x);
  }
  return out;
}
function flattenDeep(arr) {                    // iterative: no recursion limit
  const stack = [...arr], out = [];
  while (stack.length) {
    const x = stack.pop();
    if (Array.isArray(x)) stack.push(...x); else out.push(x);
  }
  return out.reverse();                        // we took items from the end
}
const nested = [1, [2, [3, [4]], 5]];
assert.deepEqual(flatten(nested, 1), nested.flat(1));   // [1, 2, [3, [4]], 5]
assert.deepEqual(flattenDeep(nested), nested.flat(Infinity));

function flattenObject(obj, prefix = '', out = {}) {     // frequent variant: dotted keys
  for (const [k, v] of Object.entries(obj)) {
    const key = prefix ? `${prefix}.${k}` : k;
    if (v && typeof v === 'object' && !Array.isArray(v)) flattenObject(v, key, out);
    else out[key] = v;
  }
  return out;
}
assert.deepEqual(flattenObject({ a: 1, b: { c: 2, d: { e: 3 } } }), { a: 1, 'b.c': 2, 'b.d.e': 3 });
```

A `bind` polyfill (and `call`)

A **polyfill** is code that implements a standard feature for environments that lack it (§10.12). `bind` must (1) fix `this`, (2) pre-fill arguments (partial application), and (3) when the bound function is used with `new`, ignore the bound `this` and construct normally. How `this` works is in §9.1–9.2.

```
Function.prototype.myBind = function (thisArg, ...preset) {
  const target = this;                                  // the function myBind was called on
  if (typeof target !== 'function') throw new TypeError('myBind must be called on a function');
  function bound(...later) {
    if (new.target) return new target(...preset, ...later);   // called with `new`
    return target.apply(thisArg, [...preset, ...later]);
  }
  return bound;
};
const user = { name: 'Ada' };
function greet(greeting, punct) { return `${greeting}, ${this.name}${punct}`; }
const hi = greet.myBind(user, 'Hi');
assert.equal(hi('!'), 'Hi, Ada!');
assert.equal(hi.call({ name: 'Bob' }, '?'), 'Hi, Ada?');   // bound this can't be overridden
function Point(x, y) { this.x = x; this.y = y; }
const p = new (Point.myBind(null, 1))(2);
assert.ok(p instanceof Point);                          // { x: 1, y: 2 }

Function.prototype.myCall = function (thisArg, ...args) {
  const ctx = thisArg == null ? globalThis : Object(thisArg);   // sloppy-mode rule
  const key = Symbol('fn');                                     // unique: can't clash
  ctx[key] = this;                                              // call it AS A METHOD of ctx
  try { return ctx[key](...args); } finally { delete ctx[key]; }
};
```

Output: `myBind: Hi, Ada! | new bound -> {"x":1,"y":2}`

`myCall` shows the classic trick: calling a function as a method of `ctx` makes `this === ctx`. (The real `call` doesn't box primitives or default to `globalThis` in strict code — that's the sloppy-mode behaviour.) Adding methods to built-in prototypes is fine in an interview but **not** in real code — it can clash with future standard methods.

Deep clone: `structuredClone` vs manual

A **shallow copy** (`{ ...obj }`, `[...arr]`, `Object.assign`) copies the top level only; nested objects are still shared. A **deep copy** copies every level. The built-in is `structuredClone` (browsers since 2022, Node 17+), which uses the *structured clone algorithm* (the one `postMessage` uses).

From `code/05_classics_data.mjs`:

```
const original = {
  when: new Date(0), tags: new Set(['a']), meta: new Map([['k', { n: 1 }]]),
  nested: { list: [1, { deep: true }] },
};
original.self = original;                                  // a cycle
const copy = structuredClone(original);                    // Date, Set, Map, cycles: all fine
structuredClone({ f() {} });                               // throws DataCloneError
structuredClone(new Point(-3));                            // becomes a plain { x: -3 }
JSON.parse(JSON.stringify({ d: new Date(0), u: undefined, m: new Map([[1, 2]]), n: NaN }));
```

Output:

```
structuredClone keeps Date/Set/Map/cycles: 1970-01-01T00:00:00.000Z [ 'a' ] true
function -> DataCloneError
class instance -> plain object: false {"x":-3}
JSON clone: { d: '1970-01-01T00:00:00.000Z', m: {}, n: null }
```

	Spread / `Object.assign`	`JSON.parse(JSON.stringify(x))`	`structuredClone(x)`	Manual `deepClone`
Depth	Top level only	Deep	Deep	Deep
`Date`	Shared	Becomes a string	✓	✓
`Map` / `Set`	Shared	Become `{}`	✓	✓
`undefined`, `NaN`, `Infinity`	✓	Dropped / `null`	✓	✓
Cycles	✓ (shared)	Throws `TypeError`	✓	✓ (with a `WeakMap`)
Functions	Shared	Dropped	Throws `DataCloneError`	Shared (as written)
Class prototype	Lost (spread gives a plain object)	Lost	Lost	Kept (as written)
Getters, property descriptors	Values copied	Values copied	Values copied	Values copied

The manual version interviewers ask for — a recursive walk that handles primitives, `Date`, `RegExp`, `Map`, `Set`, arrays, objects with their prototype, and **cycles** via a `WeakMap` of already-copied objects:

```
function deepClone(value, seen = new WeakMap()) {
  if (value === null || typeof value !== 'object') return value;   // primitives & functions
  if (seen.has(value)) return seen.get(value);                      // cycles / shared refs
  if (value instanceof Date) return new Date(value);
  if (value instanceof RegExp) return new RegExp(value.source, value.flags);
  if (value instanceof Map) {
    const m = new Map(); seen.set(value, m);                        // register BEFORE recursing
    for (const [k, v] of value) m.set(deepClone(k, seen), deepClone(v, seen));
    return m;
  }
  if (value instanceof Set) {
    const s = new Set(); seen.set(value, s);
    for (const v of value) s.add(deepClone(v, seen));
    return s;
  }
  const out = Array.isArray(value) ? [] : Object.create(Object.getPrototypeOf(value));
  seen.set(value, out);
  for (const key of Reflect.ownKeys(value)) {                       // strings AND symbols
    const desc = Object.getOwnPropertyDescriptor(value, key);
    if (desc.enumerable) out[key] = deepClone(value[key], seen);
  }
  return out;
}
```

Output: `deepClone keeps class: true | shared ref kept: true` — the copy of `new Point(-3)` still has `norm()`, and an object referenced twice in the original is one object (referenced twice) in the copy.

Which to use? `structuredClone` for data; a manual clone only to keep class instances — where a `clone()` method is usually clearer.

Deep equal

Compare two values structurally. Decisions to state out loud: `NaN` equals `NaN`? (yes, via `Object.is`); `0` vs `-0`? (not equal with `Object.is` — say which you want); do prototypes have to match? (yes, so `[1, 2]` ≠ `{0: 1, 1: 2}`); does `{a: 1}` equal `{a: 1, b: undefined}`? (no — the key sets differ).

```
function deepEqual(a, b, seen = new WeakMap()) {
  if (Object.is(a, b)) return true;                          // handles NaN and -0 vs +0
  if (typeof a !== 'object' || typeof b !== 'object' || a === null || b === null) return false;
  if (Object.getPrototypeOf(a) !== Object.getPrototypeOf(b)) return false;
  if (seen.get(a) === b) return true;                        // already comparing this pair (cycles)
  seen.set(a, b);
  if (a instanceof Date) return a.getTime() === b.getTime();
  if (a instanceof RegExp) return String(a) === String(b);
  if (a instanceof Map) {
    if (a.size !== b.size) return false;
    for (const [k, v] of a) if (!b.has(k) || !deepEqual(v, b.get(k), seen)) return false;
    return true;
  }
  if (a instanceof Set) {
    if (a.size !== b.size) return false;
    for (const v of a) if (!b.has(v)) return false;          // primitive members only (simplification)
    return true;
  }
  const ka = Reflect.ownKeys(a), kb = Reflect.ownKeys(b);
  if (ka.length !== kb.length) return false;
  return ka.every((k) => Object.hasOwn(b, k) && deepEqual(a[k], b[k], seen));
}
assert.ok(deepEqual({ a: [1, { b: NaN }] }, { a: [1, { b: NaN }] }));
assert.ok(!deepEqual({ a: 1 }, { a: 1, b: undefined }));
assert.ok(!deepEqual([1, 2], { 0: 1, 1: 2 }));
{ const x = { v: 1 }; x.me = x; const y = { v: 1 }; y.me = y; assert.ok(deepEqual(x, y)); }
```

In tests you don't write this: `assert.deepStrictEqual` does it (§16.2).

LRU cache

An **LRU (least recently used) cache** holds at most `capacity` entries; when full, it evicts the entry that was used longest ago. The textbook design is a hash map plus a doubly linked list. In JavaScript a single `Map` is enough, because a `Map` iterates in insertion order: delete-and-reinsert moves a key to the "newest" end, and `map.keys().next().value` is the oldest. Both operations are O(1).

```
class LRUCache {
  #cap; #map = new Map();
  constructor(capacity) { this.#cap = capacity; }
  get(key) {
    if (!this.#map.has(key)) return -1;
    const v = this.#map.get(key);
    this.#map.delete(key); this.#map.set(key, v);           // move to "most recent" end
    return v;
  }
  put(key, value) {
    this.#map.delete(key);                                   // updating also refreshes recency
    this.#map.set(key, value);
    if (this.#map.size > this.#cap) {
      const oldest = this.#map.keys().next().value;          // first key = least recent
      this.#map.delete(oldest);
    }
  }
  keys() { return [...this.#map.keys()]; }
}
const lru = new LRUCache(2);
lru.put('a', 1); lru.put('b', 2);
assert.equal(lru.get('a'), 1);          // a is now most recent
lru.put('c', 3);                        // evicts b
assert.equal(lru.get('b'), -1);
lru.put('a', 10);                       // update moves a to the end
assert.deepEqual(lru.keys(), ['c', 'a']);
```

If the interviewer forbids the `Map` trick, build the linked list: nodes with `prev` / `next`, a `Map` from key to node, and dummy head/tail nodes so insert and remove never special-case the ends. The LLD handbook builds the linked-list version (in Java) with eviction policies and thread safety: §20.3.

EventEmitter

The **publish/subscribe** pattern: code registers listeners for named events; `emit` calls them in registration order. Node's own `EventEmitter` is covered in §15.8; this is the interview version with `on`, `off`, `once` and `emit`.

```
class EventEmitter {
  #listeners = new Map();                                  // event -> array of fns
  on(event, fn) {
    if (!this.#listeners.has(event)) this.#listeners.set(event, []);
    this.#listeners.get(event).push(fn);
    return () => this.off(event, fn);                      // handy unsubscribe
  }
  off(event, fn) {
    const list = this.#listeners.get(event);
    if (!list) return this;
    const i = list.findIndex((l) => l === fn || l.original === fn);
    if (i !== -1) list.splice(i, 1);
    return this;
  }
  once(event, fn) {
    const wrapper = (...args) => { this.off(event, wrapper); fn.apply(this, args); };
    wrapper.original = fn;                                 // so off(event, fn) still works
    return this.on(event, wrapper);
  }
  emit(event, ...args) {
    const list = this.#listeners.get(event);
    if (!list || list.length === 0) {
      if (event === 'error') throw args[0];                // same rule as Node
      return false;
    }
    for (const fn of [...list]) fn.apply(this, args);      // copy: listeners may unsubscribe
    return true;
  }
  listenerCount(event) { return this.#listeners.get(event)?.length ?? 0; }
}
```

Output: `EventEmitter log: [ 'A:1', 'once:1', 'B:1', 'B:2' ]`

A good answer iterates over a **copy** of the listener list (a `once` listener removes itself during `emit`, which would otherwise skip the next one), lets `off` remove a `once` listener by the original function, throws on an unhandled `'error'` like Node, and returns an unsubscribe function.

`Promise.all`, `allSettled`, `race`, `any` polyfills

How the real combinators behave is in §12.8. From `code/06_promise_polyfills.mjs`:

```
function promiseAll(iterable) {
  return new Promise((resolve, reject) => {
    const items = [...iterable];                      // accept any iterable
    const results = new Array(items.length);
    let remaining = items.length;
    if (remaining === 0) return resolve(results);     // all([]) resolves immediately
    items.forEach((item, i) => {
      Promise.resolve(item).then((v) => {             // wrap non-promises too
        results[i] = v;                               // by index, not push -> keeps order
        if (--remaining === 0) resolve(results);
      }, reject);                                     // first rejection wins; later ones ignored
    });
  });
}

function promiseAllSettled(iterable) {                // never rejects
  return promiseAll([...iterable].map((item) =>
    Promise.resolve(item).then(
      (value) => ({ status: 'fulfilled', value }),
      (reason) => ({ status: 'rejected', reason }))));
}

function promiseRace(iterable) {                      // first to SETTLE, either way
  return new Promise((resolve, reject) => {
    for (const item of iterable) Promise.resolve(item).then(resolve, reject);
  });                                                 // race([]) stays pending forever
}

function promiseAny(iterable) {                       // first to FULFIL
  return new Promise((resolve, reject) => {
    const items = [...iterable], errors = new Array(items.length);
    let remaining = items.length;
    if (remaining === 0) return reject(new AggregateError([], 'All promises were rejected'));
    items.forEach((item, i) => Promise.resolve(item).then(resolve, (e) => {
      errors[i] = e;
      if (--remaining === 0) reject(new AggregateError(errors, 'All promises were rejected'));
    }));
  });
}
```

The tests run each polyfill next to the built-in:

```
const all = await promiseAll([sleep(30, 'slow'), sleep(25, 'fast'), 42]);
assert.deepEqual(all, ['slow', 'fast', 42]);           // input order, not finish order
assert.deepEqual(await promiseAll([]), []);
await assert.rejects(promiseAll([sleep(20, 'ok'), sleep(5, 'bad', true)]), /bad/);
assert.equal(await promiseRace([sleep(20, 'a'), sleep(5, 'b')]), 'b');
assert.equal(await promiseAny([sleep(5, 'e1', true), sleep(15, 'win')]), 'win');
```

Output:

```
all: [ 'slow', 'fast', 42 ] | parallel (under 55 ms)? true
allSettled: [ 1, 'rejected(x)', 'y' ]
race: b
any all-rejected: AggregateError [ 'p', 'q' ]
pool results: [ 't0', 't1', 't2', 't3', 't4' ] peak concurrency: 2
all promise asserts passed
```

Four things interviewers look for in `Promise.all`: results **by index** (not `push`, which orders by finish time); a **counter**, not `results.length` (a pre-sized array's length is already full); `Promise.resolve(item)` so plain values work; and an **empty input** that resolves immediately. A settled promise ignores later calls, so `reject` can be passed straight through.

The same file also builds on these: a **timeout** (`race` a promise against a rejecting timer), `promisify` (wrap an error-first callback API — Node ships `util.promisify`), a **concurrency-limited pool** (run at most *n* tasks at once, results in order — the peak is asserted to be 2), and **retry**. The pool core:

```
async function pool(tasks, limit) {
  const results = new Array(tasks.length);
  let next = 0;
  async function worker() {
    while (next < tasks.length) {
      const i = next++;                  // no race: nothing else runs between these lines
      results[i] = await tasks[i]();
    }
  }
  await Promise.all(Array.from({ length: Math.min(limit, tasks.length) }, worker));
  return results;
}
```

More production-ready patterns (retry with back-off, stale responses) are in §12.13.

Common mistakes

- Using arrow functions for `debounced` / `throttled` / `memoized` wrappers, which silently drops the caller's `this`.
- `Promise.all` with `results.push(v)` (wrong order) or `if (results.length === n)` (fires too early).
- Deep clone without a `seen` map — infinite recursion on the first cycle.
- `curry` that mutates a shared argument array, so partial functions interfere with each other.
- `EventEmitter.emit` iterating the live array while a `once` listener removes itself.

In the interview

30-second answer — "Implement debounce." Return a wrapper holding a timer in a closure; each call clears it and starts a new `wait`-ms timer; when it fires, call the original with the latest arguments and `this`. Add `cancel`; mention the leading-edge option.

- *Debounce vs throttle?* Debounce waits for quiet and runs once; throttle runs at a steady maximum rate during activity.
- *Why store `Promise.all` results by index?* Promises finish in any order; the result must match the input order.
- *Why is a `Map` enough for an LRU cache?* It preserves insertion order and supports O(1) delete/insert, so "move to newest" = delete + set and "oldest" = first key.

Self-check

1. What does `debounce` do with the arguments of calls that were superseded? → Nothing — only the last call's arguments are used.
2. What does `Promise.race([])` return? → A promise that never settles.
3. Why does `deepClone` call `seen.set` *before* copying children? → So a child that points back to the parent finds the copy instead of recursing forever.
4. What is `fn.length` for `(a, b = 2, ...rest) => {}`? → `1` — counting stops at the first default or rest parameter.

TL;DR

Closures hold state (`debounce` timer, `memoize` cache, `curry` arguments); forward `this` with `apply`; `bind` must handle `new`. `structuredClone` for data, manual clone for class instances, `WeakMap` for cycles. LRU = one `Map`. EventEmitter: copy the list before emitting. `Promise.all`: index, counter, `Promise.resolve`, empty input.

17.4 Fifty-three tricky interview questions

How to use this section

Each question shows the code, the **real output** (from `code/07_qa_language.js` – CommonJS in sloppy mode on purpose, so legacy behaviour is visible – and `code/08_qa_async_and_memory.mjs`), a short answer you can say out loud, and a link to the section that teaches the topic properly. Cover the answer, predict the output, then check.

17.4.1 Coercion and equality

Q1. What do `1 + '2'`, `'5' - 2`, `'3' * '4'`, `true + 1`, `[] + []`, `[] + {}` and `[1,2] + [3]` give?

```
Q1  1 + '2'                    "12"
Q1  '5' - 2                    3
Q1  '3' * '4'                  12
Q1  true + 1                   2
Q1  [] + []                    ""
Q1  [] + {}                    "[object Object]"
Q1  [1,2] + [3]                "1,23"
```

`+` first turns objects into primitives (arrays → `"1,2"`-style strings, plain objects → `"[object Object]"`); if **either** side is then a string it concatenates, otherwise it adds. `-`, `*`, `/` always convert to numbers. (At the start of a console line, `{} + []` is `0`: `{}` parses as an empty block and `+[]` is unary plus.) → §2.5, §2.12

Q2. What do `null == undefined`, `null == 0`, `null >= 0`, `'' == 0` and `'0' == false` give?

```
Q2  null == undefined          true
Q2  null == 0                  false
Q2  null >= 0                  true
Q2  '' == 0                    true
Q2  '0' == false               true
```

`==` converts types first (booleans and strings become numbers), but `null` and `undefined` are `==` only to each other, so `null == 0` is false. Relational operators have no such exception – `null` becomes `0`, so `null >= 0` is true. Use `===`; keep `x == null` for a deliberate "null or undefined" check. → §2.6

Q3. Why is `[] == ![]` true?

```
Q3  [] == ![]                  true
```

`![]` is `false` (every object is truthy). Now `[] == false`: the boolean becomes `0`, the array becomes `''` then `0`, and `0 == 0`. The same array is "equal to its own negation" – the best argument for `===`. → §2.6, §2.11

Q4. What is `typeof null`, and why?

```
Q4  typeof null                "object"
```

A bug from the first engine (1995): values carried a type tag, objects had tag 0, and `null` (the null pointer) also read as tag 0. Fixing it would break the web. Use `x === null`, and `x !== null && typeof x === 'object'` for "a real object". → §2.3

Q5. How does `NaN` behave?

```
Q5  NaN === NaN                false
Q5  [NaN].includes(NaN)        true
Q5  [NaN].indexOf(NaN)         -1
Q5  typeof NaN                 "number"
```

`NaN` (the result of failed numeric operations like `0 / 0`) is the only value not equal to itself. `indexOf` uses `===`, so it can't find it; `includes`, `Map` and `Set` use SameValueZero, where `NaN` equals `NaN`. Test with `Number.isNaN`, not the global `isNaN`, which converts first (`isNaN('x')` is `true`). → §2.10

Q6. How does `Object.is` differ from `===`?

```
Q6  Object.is(-0, 0) / -0 === 0    "false / true"
```

Only in two cases: `Object.is(NaN, NaN)` is `true` and `Object.is(0, -0)` is `false`. React uses `Object.is` to decide whether state changed. → §2.6

Q7. What does `typeof` return for arrays, functions, classes and undeclared variables?

```
Q7  typeof [] / function / class   "object / function / function"
Q7  typeof undeclaredVar           "undefined"
```

Arrays are objects — use `Array.isArray`. Classes are functions underneath. `typeof` on a name that was never declared returns `'undefined'` instead of throwing (useful for feature detection) — but see Q16 for the TDZ exception. → §2.3

Q8. Which values are falsy? Is `[]` truthy? And `new Boolean(false)`?

```
Q8  Boolean([]), Boolean('0')      "true, true"
Q8  !!new Boolean(false)           true
```

Falsy: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN` (plus the legacy `document.all`). Everything else is truthy — `[]`, `{}`, `'0'`, `'false'`, even a `Boolean` object holding `false`. Test an empty array with `arr.length === 0`. → §2.11

Q9. What does `'b' + 'a' + +'a' + 'a'` give?

```
Q9  'b' + 'a' + +'a' + 'a'         "baNaNa"
```

`+'a'` is unary plus: it converts `'a'` to a number, which is `NaN`; concatenation turns it into the text `"NaN"`. → §2.5

Q10. Why does `['1', '7', '11'].map(parseInt)` give `[1, NaN, 3]`?

```
Q10 ['1','7','11'].map(parseInt)   [ 1, NaN, 3 ]
Q10 ['1','7','11'].map(Number)     [ 1, 7, 11 ]
```

`map` passes `(value, index, array)`, and `parseInt`'s second parameter is the **radix**: `parseInt('1', 0)` (0 = default base 10 → 1), `parseInt('7', 1)` (invalid base → `NaN`), `parseInt('11', 2)` (binary → 3). Fix: `map(Number)` or `map((s) => parseInt(s, 10))`. → §7.5, §2.10

Q11. Why does `[10, 1, 2].sort()` give `[1, 10, 2]`?

```
Q11 [10, 1, 2].sort()              [ 1, 10, 2 ]
```

Without a comparator, `sort` compares elements as strings, and `'10' < '2'` because `'1' < '2'`. Pass `(a, b) => a - b`. → §7.6

Q12. What's the difference between `||` and `??`?

```
Q12 0 || 'x' / 0 ?? 'x'            "x / 0"
```

`a || b` falls back to `b` when `a` is **any falsy** value, so it wrongly replaces a legitimate `0`, `''` or `false`. `a ?? b` (nullish coalescing, ES2020) falls back only when `a` is `null` or `undefined`. Use `??` for defaults. → §2.7

Q13. Why is `{} === {}` false?

```
Q13 {} === {}                    false
```

Objects (including arrays and functions) are compared by **reference** — "is this the same object in memory?" — not by content. Two literals create two objects. To compare content, use a deep-equal (§17.3). → §2.4

Q14. Why is `0.1 + 0.2 !== 0.3`? And why is `Math.max() < Math.min()`?

```
Q14 0.1 + 0.2 === 0.3            false
Q14 Math.max() < Math.min()      true
```

0.1 and 0.2 have no exact binary (IEEE 754) form, so the sum is `0.30000000000000004` — compare with a tolerance or use integers (§17.1). With no arguments `Math.max()` is `-Infinity` (the identity for "max") and `Math.min()` is `Infinity`. → §2.10

17.4.2 Scope, hoisting and closures

Q15. What are the differences between `var`, `let` and `const`?

	`var`	`let`	`const`
Scope	Function	Block `{ }`	Block `{ }`
Before the declaration line	`undefined` (hoisted and initialised)	`ReferenceError` (TDZ)	`ReferenceError` (TDZ)
Redeclare in the same scope	Allowed	`SyntaxError`	`SyntaxError`
Reassign	Yes	Yes	No (`TypeError`)
At top level of a script	Becomes a property of `globalThis`	Doesn't	Doesn't

`const` stops *reassignment* of the variable, not mutation of the object it points to (Q44). Default to `const`, use `let` when you must reassign, avoid `var`. → §2.1

Q16. What happens when you read a variable before its declaration?

```
Q16 var read before declaration    "undefined"
Q16 let read before declaration    "ReferenceError: Cannot access 'tdz' before initialization"
Q16 typeof in the TDZ              "ReferenceError: Cannot access 'inTdz' before initialization"
```

All declarations are **hoisted** (registered on entering the scope); `var` is also initialised to `undefined`. `let`, `const` and `class` stay uninitialised until their line runs — the **temporal dead zone (TDZ)** — and any access throws, even `typeof`. → §2.2

Q17. Can you call a function before it's defined?

```
Q17 declaration hoisted          "works"
Q17 expression not hoisted       "TypeError: late is not a function"
```

A function **declaration** is hoisted with its body. A function **expression** in a `var` hoists only the variable (`undefined`), so calling it throws `TypeError`; with `let`/`const` it's a TDZ `ReferenceError`. → §4.2

Q18. What do these loops print?

```
for (var i = 0; i < 3; i++) setTimeout(() => out.var.push(i), 0);
for (let j = 0; j < 3; j++) setTimeout(() => out.let.push(j), 0);
for (var k = 0; k < 3; k++) ((k2) => setTimeout(() => out.fixed.push(k2), 0))(k);
```

```
Q18 closures in loops            {"var":[3,3,3],"let":[0,1,2],"fixed":[0,1,2]}
```

The callbacks run after the loop. `var` has **one** `i` for the whole function — `3` by then. `let` in a `for` header creates a **fresh binding per iteration**. The pre-ES2015 fix was an IIFE copying the value into a parameter. → §4.7

Q19. What is a closure? Show one.

```
const makeCounter = () => { let n = 0; return () => ++n; };
const c1 = makeCounter(), c2 = makeCounter();
c1(); c1();
```

```
Q19 counter via closure            "c1: 3, c2: 1"
```

A **closure** is a function plus the variables of the scope where it was created, kept alive and updatable after the outer function returns. Each `makeCounter()` call creates a new `n`, so `c1` and `c2` are independent. Uses: private state, factories, `debounce` / `memoize` (§17.3). → §4.7

Q20. What's wrong with `let a = b = 5`?

```
(function () { let a = b = 5; })();
```

```
Q20 let a = b = 5                  "outside: typeof a=undefined, typeof b=number"
```

It parses as `b = 5; let a = b;`. `a` is local, but `b` was never declared, so in sloppy mode the assignment creates an **accidental global**. In strict mode it throws `ReferenceError` (Q22). → §4.5

Q21. Is a `var` inside an `if` block visible outside it?

```
Q21 var in block leaks             "v=1, typeof l=undefined"
```

Yes: `var` ignores blocks and belongs to the whole function. `let` / `const` are confined to the block. → §4.5

Q22. What does strict mode change?

```
Q22/Q23 strict: lost this      TypeError: Cannot read properties of undefined (reading 'name')
Q22 strict: undeclared assign  ReferenceError: undeclaredThing is not defined
```

Strict mode (automatic in ES modules and class bodies) turns silent mistakes into errors: undeclared assignments and writes to read-only properties throw, `this` in a plain call is `undefined` instead of `globalThis`, and duplicate parameter names and `with` are syntax errors. → §1.8

17.4.3 `this`, arrow functions, `call` / `apply` / `bind`, `new`

Q23. Why does a method lose `this` when you pass it around?

```
const f = user.regular; f();
```

```
Q23 method call                    "Ada"
Q23 extracted method (sloppy)      undefined
```

`this` is decided by **how** a function is called. `user.regular()` sets `this = user`; a plain `f()` gives `undefined` in strict code (→ `TypeError`, Q22) or `globalThis` in sloppy code (→ `undefined` name). It bites with callbacks like `setTimeout(user.regular)`. Fix: `user.regular.bind(user)` or `() => user.regular()`. → §9.1

Q24. How do arrow functions differ from regular functions?

```
Q24 arrow in object literal        "object"
```

	Regular function	Arrow function
`this`	Set by the call	Taken from the surrounding scope (lexical); `call` / `bind` can't change it
`arguments`	Has its own	None (uses the outer one); use rest `...args`
`new`	Can be a constructor	`TypeError` – no `prototype`
Good for	Methods, constructors	Callbacks inside methods

So an arrow used as an object-literal *method* doesn't see the object: its `this` is the outer scope's – here CommonJS's `module.exports`; in an ES module, `undefined`. → §4.3

Q25. Inside a method, what is `this` in a nested regular function vs a nested arrow?

```
delayed() { return [function () { return this === globalThis; }(), (() => this.name)()]; }
```

```
Q25 inner function vs arrow        [ true, 'Ada' ]
```

The inner regular function is called plainly (default `this`: global object in sloppy mode, `undefined` in strict); the arrow inherits the method's `this` – why arrows suit callbacks like `setTimeout(() => this.save(), 100)` inside methods. → §9.1

Q26. What's the difference between `call`, `apply` and `bind`?

```
Q26 call / apply / bind            "Hi Lin! | Yo Lin? | Hey Lin."
```

All three set `this`. `call(thisArg, a, b)` calls now with arguments listed; `apply(thisArg, [a, b])` calls now with an array; `bind(thisArg, a)` **returns a new function** with `this` (and optionally leading arguments) fixed, to call later. Memory aid: **A**pply = **A**rray. → §9.2

Q27. What happens if you `bind` twice?

```
Q27 bind twice                     1
```

The first `bind` wins: a bound function ignores any later `this` from `bind`, `call` or `apply` (the only exception is `new`). → §9.2

Q28. What does `new` do? What if the constructor returns something?

```
Q28 new returns object?            "{\"b\":2}"
Q28 new returns primitive?         "{\"a\":1}"
```

`new F(args)`: (1) creates an empty object whose prototype is `F.prototype`; (2) calls `F` with `this` set to that object; (3) returns that object – **unless** `F` explicitly returns an *object*, which then replaces it. Returned primitives are ignored. → §9.4

17.4.4 Prototypes and classes

Q29. What's the difference between `prototype` and `__proto__`?

```
Q29 proto links                    "true true true"
Q29 own vs inherited               "true false true"
```

Every object has an internal `[[Prototype]]` link (read with `Object.getPrototypeOf`; `__proto__` is the legacy accessor); missed lookups continue along this **prototype chain**. `F.prototype` is an ordinary property of **functions**: the object `new F()` installs as the instance's `[[Prototype]]`. So methods live on `Animal.prototype` (shared) and data on the instance (`Object.hasOwn(rex, 'speak')` is false, `'speak' in rex` is true). → §9.3

Q30. How does `instanceof` work? Can it be fooled?

```
Q30 instanceof after proto swap    true
```

`x instanceof F` walks `x`'s prototype chain looking for `F.prototype`. It doesn't check how `x` was created, so a plain object whose prototype was set to `Animal.prototype` passes. It also fails across realms (an array from an iframe isn't `instanceof Array` here — use `Array.isArray`). → §9.7

Q31. How are classes different from constructor functions?

```
Q31 class before declaration       "ReferenceError: Cannot access 'Later' before initialization"
Q31 class without new              "TypeError: Class constructor K cannot be invoked without 'new'"
```

Mostly syntax over prototypes, but: class declarations are in the TDZ until defined; calling without `new` throws; bodies are strict; methods are non-enumerable; and classes add real features — `#private`, `static` blocks, `super`. → §9.5

Q32. What is `Object.create(null)` for?

```
Q32 Object.create(null)            "undefined true"
```

It creates an object with **no prototype** — no inherited `toString`, `constructor` or `__proto__` — a safe "dictionary" where keys like `'constructor'` from user input can't collide with built-ins. A `Map` is usually the better choice today. → §9.3

Q33. How do `#private` fields compare with closure-based privacy?

```
class Wallet { #cents = 0; add(c) { this.#cents += c; return this.#cents; } }
```

```
Q33 #private vs closure            "10 undefined 0 {}"
```

`#cents` is truly private — invisible to `w.cents`, `Object.keys` and `JSON.stringify`, and a *syntax* error outside the class body. Closure privacy also hides state, but every instance needs its own method copies; `#` fields keep methods shared on the prototype. → §9.5

Q34. What happens if a subclass constructor uses `this` before `super()`?

```
Q34 this before super()            "ReferenceError: Must call super constructor in derived class before
accessing 'this' or returning from derived constructor"
```

In a derived class the **parent** constructor creates the object, so `this` doesn't exist until `super(...)` returns. → §9.6

17.4.5 The event loop and async code

For all of these, apply the rules from §12.2–12.3: run all synchronous code first; then drain the **microtask** queue (promise callbacks, code after `await`, `queueMicrotask`) completely; then run **one task** (timer, I/O, event); drain microtasks again; repeat.

Q35. The classic.

```
log('1');
setTimeout(() => log('2'), 0);
Promise.resolve().then(() => log('3'));
log('4');
```

```
Q35 classic                    1 4 3 2
```

`1` and `4` are synchronous. The `.then` callback is a microtask, so it runs as soon as the script finishes; the timer is a task and waits for the next loop turn — even with a 0 ms delay. → §12.2

Q36. `async` / `await` ordering.

```
async function a() { log('a1'); await b(); log('a2'); }
async function b() { log('b'); }
log('s1');
setTimeout(() => log('t'), 0);
a();
new Promise((res) => { log('p'); res(); }).then(() => log('then'));
log('s2');
```

```
Q36 async/await                s1 a1 b p s2 a2 then t
```

An `async` function runs **synchronously until its first `await`** (`a1`, `b`); the rest (`a2`) becomes a microtask. The promise **executor** is synchronous too (`p`). After `s2`, microtasks run in queue order — `a2`, then `then` — and the timer last. → §12.3, §12.9

Q37. `process.nextTick`, promises and `setImmediate` in Node.

```
setImmediate(() => q.push('immediate'));
Promise.resolve().then(() => q.push('then'));
process.nextTick(() => q.push('nextTick'));
q.push('sync');
```

```
Q37 CommonJS main                  sync nextTick then immediate
Q37 ESM (after await)          sync then nextTick immediate
```

Node normally drains `nextTick` **before** promise microtasks (the CommonJS result). Inside an ES module (here after a top-level `await`) the code already runs in a promise job, so promises drain first. `setImmediate` runs later, in the *check* phase. Don't rely on the `nextTick`/promise order; prefer `queueMicrotask`. → §12.4

Q38. When does a promise executor run, and what if you resolve twice?

```
Q38 executor + settle once     executor after first
```

The function passed to `new Promise(...)` runs **synchronously**, right away. A promise settles **once**: the first `resolve`/`reject` wins, later calls are ignored silently. → §12.7

Q39. Why doesn't this `try` / `catch` catch the error?

```
try { setTimeout(() => { throw new Error('boom'); }, 0); } catch { /* never runs */ }
try { await Promise.reject(new Error('rejected')); } catch (e) { /* runs */ }
```

```
Q39 errors in callbacks        uncaught: boom | await caught: rejected
```

`try` only covers code running *while it's on the stack*. The timer callback runs later, so its error is uncaught (the demo shows it via `process.on('uncaughtException')`; normally Node would crash). With `await`, the rejection is thrown back into the function at the `await`, so `try` / `catch` works. → §11.5, §12.10

Q40. Does `await` inside `forEach` wait?

```
[20, 10].forEach(async (ms) => { await wait(ms); log.push(ms); });
log.push('forEach returned');
for (const ms of [20, 10]) { await wait(ms); log.push(`for-of ${ms}`); }
```

```
Q40 await in forEach           forEach returned | 10 | 20 | for-of 20 | for-of 10
```

No. `forEach` ignores the promise each async callback returns, so it returns immediately and callbacks finish in timer order. Use `for…of` + `await` for sequential work, `await Promise.all(arr.map(async …))` for parallel. → §12.9

Q41. Two independent `await`s — how long do they take?

```
Q41 sequential vs parallel     sequential >= 80ms: true, parallel < 80ms: true
```

`await wait(40); await wait(40);` takes ~80 ms because the second doesn't start until the first finishes. Starting both and awaiting `Promise.all([...])` takes ~40 ms. Start independent work first, await later. → §12.9

Q42. `Promise.all` vs `allSettled` vs `race` vs `any`?

Combinator	Fulfils when	Rejects when
`all`	All fulfil (array of values, input order)	The first rejection
`allSettled`	All settle (array of `{status, value/reason}`)	Never
`race`	The first to settle fulfils	The first to settle rejects
`any`	The first fulfils	All reject (`AggregateError`)

Implementations and tests: §17.3. → §12.8

17.4.6 Objects, arrays and copies

Q43. Shallow vs deep copy — what does this print?

```
const a = { n: { v: 1 } }; const b = { ...a }; b.n.v = 2; a.n.v;
```

```
Q43 spread copy is shallow        2
```

Spread copies only the top level; `b.n` and `a.n` are the same nested object. For a deep copy use `structuredClone` (or the manual clone, §17.3). → §5.4

Q44. Can you change a `const` object? Does `Object.freeze` make it immutable?

```
Q44 const + mutate                2
Q44 freeze is shallow             "2 undefined"
```

`const` protects the **binding**, not the object. `Object.freeze` blocks adding, removing and changing *own* properties (silently in sloppy mode, `TypeError` in strict) but is **shallow** — nested objects stay mutable; deep-freeze by recursing. → §5.6

Q45. What happens when you use an object as an object key?

```
const o = {}; o[{}] = 'a'; o[{ id: 2 }] = 'b'; o[1] = 'c';
```

```
Q45 object keys become strings    "[\"1\",\"[object Object]\"] b"
```

Property keys are strings (or symbols). Both objects convert to the same string `"[object Object]"`, so the second assignment overwrites the first; `1` becomes `"1"`. Use a `Map` for object keys. → §5.2, §8.1

Q46. What do `delete arr[i]` and `arr.length = n` do?

```
Q46 array holes                   "3 [1,null,3] false"
Q46 length truncates              "[1]"
Q46 length of sparse              6
```

`delete` removes the element but leaves a **hole** — the length stays 3 (JSON shows the hole as `null`). Use `splice` to remove. Setting `length` smaller truncates the array (`arr.length = 0` empties it in place). Assigning `a[5]` on an empty array makes `length` 6 with five holes. → §7.3

Q47. Is JavaScript pass-by-value or pass-by-reference?

```
const o = { v: 1 };
(function (p) { p = { v: 99 }; })(o);   // reassign the parameter
(function (p) { p.v = 2; })(o);          // mutate through it
o.v;
```

```
Q47 reassign param                2
```

Always **by value** — but an object's value *is a reference* ("call by sharing"): mutating through the parameter changes the caller's object; reassigning it changes only the local variable. → §2.4

Q48. When should you use a `Map` instead of an object?

Use a `Map` for dynamic keys added and removed at runtime, non-string keys, frequent size checks and guaranteed insertion order; use a plain object for fixed-shape records (a user, a config) and JSON. Objects also have inherited keys (`'toString' in {}` is true) — a source of bugs with user-supplied keys. → §7.8, §8.1

Q49. What does `JSON.stringify` drop?

```
Q49 JSON drops                    "{\"n\":null,\"d\":\"1970-01-01T00:00:00.000Z\"}"
Q49 JSON array holes              "[null,null]"
```

Object properties holding `undefined`, a function or a symbol are **omitted** (in arrays they become `null`); `NaN` / `Infinity` → `null`; dates → ISO strings; `Map` / `Set` → `{}`; `BigInt` and cycles throw. → §5.12

Q50. `for…in` vs `for…of`?

```
Q50 for...in vs for...of          "in: 0,1,extra | of: a,b"
```

`for…in` loops over **enumerable property keys** (strings), including inherited ones and extras like `extra` — meant for plain objects. `for…of` loops over the **values** of any *iterable*; use it for arrays. → §3.10

17.4.7 Garbage collection and memory leaks

Q51. How does JavaScript free memory?

Automatically, with a **garbage collector (GC)**. An object stays alive while it is **reachable** from a *root* (globals, the call stack, active closures, pending callbacks). Unreachable objects are freed even if they reference each other — **mark-and-sweep** marks everything reachable and sweeps the rest, so cycles are fine. V8 is **generational**: a small *young generation* is collected often and cheaply (most objects die young), survivors move to an *old generation* collected less often, much of it on helper threads. Normal code can't (and needn't) trigger GC. → §8.3

Q52. What causes memory leaks in JavaScript, and how do you fix them?

A **leak** is memory that is still reachable but no longer needed. The usual causes:

Cause	Example	Fix
Forgotten listeners and timers	`setInterval` or `addEventListener` never removed; the callback's closure keeps data alive	`clearInterval`, `removeEventListener`, `AbortController` signal, React effect cleanup
Unbounded caches and collections	A `Map` from request/DOM objects to data that only grows	Size limit (LRU), or a `WeakMap` keyed by the object
Closures capturing big data	A long-lived callback references a huge array it doesn't need	Copy only what's needed into the closure; set big references to `null`
Detached DOM nodes	A removed element still referenced from a JS variable or array	Drop the reference when the element is removed
Accidental globals	`x = …` without declaration (sloppy mode)	Strict mode / modules

The demo stores ~8 KB of data per short-lived "request" object in a `Map` and in a `WeakMap`, then forces a GC (only possible with a V8 flag, used here for the demonstration):

```
Q52 Map vs WeakMap cache       Map retains > 10 MB: true, WeakMap retains < 2 MB: true
```

The `Map` keeps every key object and its data alive forever; the `WeakMap` lets both be collected once the request object is unreachable. Find leaks with DevTools' **Memory** panel: take heap snapshots before and after an action and compare what grew. → §4.7, §8.3, §11.9

Q53. What are `WeakMap`, `WeakRef` and `FinalizationRegistry` for?

```
Q53 WeakRef before             true
Q53 WeakRef after gc           collected
```

`WeakMap` / `WeakSet` hold keys **weakly** – an entry vanishes with its key, ideal for metadata or caches on objects you don't own. A `WeakRef` points to an object without keeping it alive (`deref()` → the object or `undefined`); `FinalizationRegistry` calls back some time after collection. GC timing is **not guaranteed**, so use these only for optional optimisations, never program logic. → §8.3

In the interview

30-second answer – "`this`, closures and the event loop in one breath?" `this` comes from the call site (method, plain call, `new`, `call` / `apply` / `bind`) except in arrows, which take it lexically. A closure is a function plus its creation scope's variables, kept alive with it. The event loop runs sync code, then all microtasks, then one task, repeat – so promise callbacks beat `setTimeout(0)`.

Self-check

1. `console.log([] + null + 1)`? → `"null1"`: `[]` becomes `''`, then string concatenation.
2. Why does `for (var i…) setTimeout(() => console.log(i))` print the final value? → One shared `var` binding; `let` creates one per iteration.
3. In `async function f() { console.log(1); await null; console.log(2); } f(); console.log(3);` – the order? → `1 3 2`.
4. `typeof typeof 1`? → `'string'` (`typeof 1` is the string `'number'`).
5. Does `Object.freeze` stop `obj.nested.x = 1`? → No, freeze is shallow.

TL;DR

Use `===`; know the falsy list, `typeof null`, `NaN` rules and how `+` concatenates. `var` is function-scoped and hoisted as `undefined`; `let` / `const` are block-scoped with a TDZ; `let` in loops gives one binding per iteration. `this` depends on the call; arrows inherit it; `bind` wins once; `new` returns the new object unless you return an object. Classes are prototypes plus TDZ, strictness and `#private`. Sync → microtasks → one task.

Spread is shallow; `structuredClone` is deep. Leaks come from things that stay reachable — listeners, timers, caches, closures; `WeakMap` and cleanup fix them.

Chapter 17 TL;DR

- **Toolkit:** `push` / `pop` stack; head-index queue (no `shift` in hot loops); `Array.from` grids; `Map` / `Set` with string keys for pairs; `sort` needs a comparator, is stable, mutates (`toSorted` doesn't); floats — tolerance, `Math.trunc`, `%` sign, safe to $2^{53} - 1$, then `BigInt`; build strings with `join`.
- **Patterns (21):** two pointers, sliding window, hashing, stack/monotonic stack, prefix sums, intervals (sort + sweep), heap (hand-written `MinHeap`, O(log n) push/pop; two heaps for a running median; k-way merge), binary search (on data or the answer), BFS (fewest steps, mark on enqueue), DFS (iterative for deep inputs), DP with memo, backtracking (choose/explore/un-choose, copy results), linked lists (dummy head, reverse, k-th from end), fast & slow pointers (cycle start, middle), trees (validate with ranges, LCA, serialize), tries (prefix, wildcard, board words), Kahn's topological sort and union-find, greedy (with a proof), quickselect / Dutch flag / merge sort, bit manipulation (32-bit signed, `>>> 0`), math & geometry (spiral, overflow, gcd-reduced slopes).
- **Classics:** `debounce` (clear + restart a timer) vs `throttle` (at most once per interval, leading + trailing); `curry` via `fn.length`; `compose` = `reduceRight`, `pipe` = `reduce`; `memoize` with a `Map`; `flatten` recursive or with a stack; `bind` must support `new`; `structuredClone` vs manual `deepClone` with a `WeakMap`; `deepEqual` with `Object.is`; LRU = one `Map`; EventEmitter copies listeners before emitting; `Promise.all` = index + counter + `Promise.resolve` + empty case.
- **Q&A:** coercion and `==` rules, `typeof null`, `NaN`, `??` vs `||`; hoisting and TDZ, closures in loops, accidental globals, strict mode; `this` rules, arrows, `call` / `apply` / `bind`, `new`; prototype chain, `instanceof`, classes vs functions, `#private`; event-loop order (sync → microtasks → task), `nextTick`, errors in callbacks, `await` in `forEach`, parallel awaits; shallow vs deep copy, `freeze`, keys as strings, holes, call by sharing, JSON; GC by reachability, leaks and weak references.

Next: the cheat sheets and glossary — Chapter 18.